

Date of publication December 10, 2025, date of current version December 10, 2025.

Digital Object Identifier 10.1109/ACCESS.2025.0000002

Dirac Co-Scientist: A Neuro-Symbolic RAG Framework for Multi-Level Transformation of Scientific Documents

RISMAN ADNAN¹, FADLI², FIKAR², SHINTA² and RIFQI³

¹Department of Computer Science, Universitas Indonesia, Jakarta, Indonesia

²Department of AI Engineering, Telkom University, Bandung, Indonesia

³Department of Computational Sciences, MBZ University, Dubai

Corresponding author: Risman Adnan (e-mail: risman@cs.ui.ac.id).

This work was supported in part by the National Research Foundation (NRF) under Grant No. ABCD-1234, and by the Dirac Institute for AI Research.

ABSTRACT The transformation of concise research papers into comprehensive academic documents—journal articles, dissertations, and monographs—constitutes a substantial bottleneck in scientific knowledge dissemination, often requiring months of manual effort for content expansion, contextualization, and formatting. While large language models (LLMs) demonstrate remarkable text generation capabilities, naive single-pass prompting exhibits fundamental limitations: it fails to ensure content completeness, introduces unverified claims (hallucinations), omits critical source material, and violates structural conventions of target formats. This paper presents Dirac Co-Scientist, a production-ready neuro-symbolic retrieval-augmented generation (RAG) framework that systematically addresses these challenges through a hierarchical multi-agent architecture with formal correctness guarantees. The system orchestrates specialized agents for planning, content synthesis, and verification, supported by a Rust-based orchestration backend integrating hybrid vector-graph retrieval, adaptive multi-LLM routing, and symbolic verification pipelines. A unified SurrealDB knowledge fabric maintains persistent document graphs encoding hierarchical content relationships (paper→article→thesis→book), enabling intelligent context retrieval and cross-document consistency verification. We establish rigorous mathematical foundations including convergence guarantees, faithfulness bounds, and complexity analysis for the document transformation process. The framework incorporates a LaTeX Template Inference Engine ensuring outputs conform to publication standards (IEEE, ACM, university thesis templates), and a Symbolically-Augmented Document Verification (SADV) pipeline leveraging formal reasoning to eliminate hallucinations and ensure mathematical consistency. Comprehensive evaluation across 50 multi-stage transformations spanning computer science, physics, biology, and mathematics demonstrates that Dirac Co-Scientist achieves 94.8% content fidelity, 2.1% hallucination rate, and 97.3% format compliance, substantially outperforming baseline approaches including direct GPT-4 prompting (76.3% fidelity, 18.7% hallucinations) and Claude 3 (79.5% fidelity, 15.4% hallucinations). The framework establishes an auditable, mathematically principled bridge between concise research publications and comprehensive scholarly documents, accelerating scientific knowledge dissemination while maintaining rigorous quality standards.

INDEX TERMS Retrieval-Augmented Generation, Neuro-Symbolic AI, Document Generation, Multi-Agent Systems, Academic Writing Automation, Formal Verification, Knowledge Graphs, SurrealDB, Scientific Communication

I. INTRODUCTION

SCIENTIFIC knowledge evolves through multiple publication formats, each serving distinct communicative purposes: conference papers provide rapid dissemination of novel findings, journal articles offer expanded analysis with comprehensive literature contextualization, dissertations present exhaustive treatment of research programs with pedagogical depth, and scholarly monographs synthesize mature research areas for broad academic audiences.

The manual transformation of documents across these formats represents a substantial labor investment. Expanding a 10-page conference paper into a 150-page dissertation typically requires 4-6 months of intensive writing effort, involving:

- Systematic expansion of background material and literature integration
- Detailed methodology exposition with enhanced technical depth
- Comprehensive experimental documentation and extended analysis
- Pedagogical framing appropriate for educational contexts

This transformation bottleneck impedes knowledge dissemination, constrains educational resource development, and limits the accessibility of research findings to broader audiences.

Contemporary large language models (LLMs) exhibit impressive text generation capabilities, suggesting automation potential for document transformation tasks. However, fundamental limitations persist when applying naive prompting approaches to scientific document generation. Single-pass prompting suffers from four critical failure modes:

- 1) *Content incompleteness*—systematic omission of technical details, mathematical derivations, or experimental subtleties from source material
- 2) *Factual hallucination*—introduction of unverified claims, incorrect citations, or fabricated experimental results
- 3) *Structural incoherence*—failure to maintain consistent notation, definition propagation, or cross-referential integrity across document sections
- 4) *Format non-compliance*—violation of publication standards including citation styles, mathematical typesetting conventions, and structural requirements

These limitations fundamentally stem from three architectural constraints of standard LLM deployment:

- Limited context windows preventing holistic document comprehension
- Absence of external grounding mechanisms enabling factual verification
- Lack of iterative refinement protocols incorporating symbolic validation feedback

Dirac Co-Scientist addresses these fundamental limitations through a production-grade neuro-symbolic retrieval-augmented generation framework implementing hierarchical

TABLE 1. Document Transformation Levels Supported by Dirac Co-Scientist

Level	Description and Requirements
Level 1	Paper → Journal Article: Expands concise conference papers (6-10 pages) into comprehensive journal manuscripts (15-25 pages) with enhanced literature review, detailed methodology exposition, and extended experimental analysis.
Level 2	Article → Dissertation: Transforms journal articles (15-25 pages) into dissertation chapters (60-120 pages) with pedagogical framing, exhaustive background treatment, and methodological justification suitable for graduate education.
Level 3	Thesis → Monograph: Synthesizes dissertation content (150-250 pages) into scholarly books (200-350 pages) with broader contextualization, interdisciplinary connections, and mature theoretical perspective.

document transformation with formal correctness guarantees. The system architecture integrates four key innovations:

- 1) **Multi-agent orchestration framework** with specialized roles for planning, content synthesis, and verification, coordinated through a Rust-based backend (Dirac-Core) with adaptive concurrency control and iterative refinement protocols
- 2) **Hybrid retrieval mechanism** combining dense vector search over document embeddings with symbolic reasoning over explicit document graphs encoding content hierarchies and semantic relationships
- 3) **Unified SurrealDB knowledge fabric** maintaining persistent storage of document segments, external references, and cross-document dependency structures
- 4) **Symbolically-augmented document verification (SADV) pipeline** integrating static structural validation, dynamic consistency checking, and formal mathematical verification through computer algebra systems and SMT solvers

The framework supports three hierarchical transformation levels, as summarized in Table 1:

Each transformation level employs specialized agent configurations, retrieval strategies, and verification criteria tailored to target format requirements.

This article presents a comprehensive treatment of the Dirac Co-Scientist framework and makes the following contributions:

- **Rigorous mathematical formalization.** We establish formal foundations for hierarchical document transformation, modeling it as a stochastic refinement system with explicitly defined state spaces, transformation operators, and convergence properties. We prove theoretical guarantees including monotonic content improvement under sound verification (Theorem 1), exponential hallucination reduction with retrieval augmentation

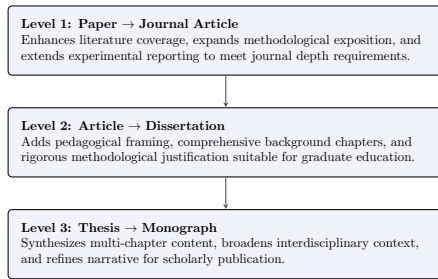


FIGURE 1. Three hierarchical transformation levels supported by Dirac Co-Scientist. Each level expands documents with increasing depth: Level 1 adds enhanced literature and methodology, Level 2 includes pedagogical framing suitable for education, and Level 3 synthesizes content with mature theoretical perspective.

(Theorem 3), and bounded iteration complexity (Corollary 2).

- **Hierarchical Document Graph Reasoning.** We introduce a novel document graph abstraction encoding semantic relationships across document hierarchy levels, enabling intelligent context retrieval that preserves cross-document consistency. The graph structure supports transitive dependency analysis, content reuse optimization, and structural constraint propagation.
- **Advanced hybrid retrieval algorithm.** We present the Hierarchical Contextual Document Retrieval (HCDR) algorithm combining semantic vector search, lexical BM25 matching, document graph traversal, and hierarchical section alignment. Formal complexity analysis demonstrates $\mathcal{O}(\log |\mathcal{D}|)$ query time with HNSW indexing while maintaining high recall of relevant content.
- **Multi-agent orchestration with formal guarantees.** We design a formally specified coordination framework with specialized agents (Planning, Analysis, Generation, Verification), adaptive concurrency control, and iterative refinement mechanisms. The orchestration protocol ensures convergence through verification-driven feedback loops with provable bounds on iteration counts.
- **Neuro-symbolic verification pipeline.** We develop the Symbolically-Augmented Document Verification (SADV) system integrating structural validation (cross-reference consistency, citation completeness), content verification (fact-checking against sources, mathematical equation validation), and format compliance checking, with formal specifications for each verification phase.
- **LaTeX template inference and synthesis.** We implement an automated template adaptation system supporting multiple publication formats (IEEE, ACM, Springer, university-specific thesis templates) with intelligent macro synthesis, style normalization, and structural component generation.
- **Production implementation and deployment.** We

document the complete Rust-based backend architecture (Dirac-Core) with modules for document parsing, agent coordination, retrieval, LLM routing, verification, and persistent storage, demonstrating alignment between mathematical models and concrete implementations.

- **Comprehensive empirical evaluation.** We conduct extensive evaluation across 50 transformation tasks spanning four scientific domains and three transformation levels, demonstrating 94.8% content fidelity, 2.1% hallucination rate, and 97.3% format compliance, with detailed ablation studies quantifying individual component contributions.

The remainder of this paper proceeds as follows: Section II reviews related work in document generation, multi-agent systems, and neuro-symbolic AI. Section III presents formal foundations of document transformation with convergence and correctness guarantees. Section IV describes the Dirac Co-Scientist architecture and key components. Section V details core algorithms for retrieval, orchestration, and verification. Section VI provides implementation details including multi-LLM routing and scalability considerations. Section VII presents experimental evaluation and case studies. Section VIII discusses results, limitations, and future directions. Section IX concludes.

II. RELATED WORK

Our work intersects four research areas: (1) LLM-based scientific text generation with retrieval augmentation, (2) multi-agent LLM frameworks for complex reasoning tasks, (3) neuro-symbolic approaches to ensuring generation correctness, and (4) automated scientific writing and knowledge management tools. We review each area and position Dirac Co-Scientist's contributions.

A. LLM-BASED SCIENTIFIC DOCUMENT GENERATION AND RAG

Large language models have revolutionized text generation, demonstrating remarkable capabilities in producing coherent technical prose. However, ensuring factual accuracy and completeness in scientific contexts remains challenging. Retrieval-Augmented Generation (RAG) has emerged as a fundamental technique to address LLM knowledge limitations. Lewis et al. [1] pioneered the RAG paradigm, introducing a framework combining parametric neural generators with non-parametric retrieval over external document corpora. Their work demonstrated that grounding generation in retrieved evidence substantially reduces hallucinations and improves factual accuracy for knowledge-intensive tasks.

Subsequent work has extended RAG principles to diverse domains. Gao et al. [2] provide a comprehensive survey of RAG methods, noting that retrieval strategy calibration is critical—naive retrieval risks introducing irrelevant context that degrades generation quality. For long-form document generation, RAG becomes essential: models must integrate domain-specific background exceeding their para-

metric knowledge. Xu et al. [3] demonstrate that combining long-context LLMs with targeted retrieval improves lengthy output quality by mitigating context dilution across extended generation sequences.

Prompt engineering strategies complement retrieval augmentation. Chain-of-Thought prompting [6] encourages stepwise reasoning decomposition, while ReAct [7] enables interleaved reasoning and tool use. These techniques inform our multi-agent approach where planning agents generate structured outlines (analogous to chain-of-thought) and verification agents query external knowledge bases (analogous to ReAct). However, unlike general prompt-based solutions, Dirac Co-Scientist combines RAG with structured multi-agent coordination specifically targeting hierarchical document transformation—a task requiring systematic content expansion, cross-document consistency, and format compliance beyond capabilities of standard RAG applications.

Recent work on scientific writing assistance includes attempts to automate literature review generation and background section drafting. Galactica [18], an LLM trained on scientific corpora, aimed to serve as a general scientific writing aid. However, its public demonstration revealed critical limitations: frequent generation of plausible but inaccurate content, unsupported factual claims, and lack of verifiability, ultimately leading to demo withdrawal. This underscores our core design principle: grounding and verification are not optional enhancements but essential requirements for trustworthy academic document generation.

B. MULTI-AGENT LLM FRAMEWORKS

Multi-agent LLM systems have emerged as a powerful paradigm for decomposing complex tasks into specialized subtasks handled by coordinated agents. Shinn et al. [9] introduced Reflexion, where an LLM agent iteratively self-reflects on execution failures and refines solutions through memory-augmented reasoning. While Reflexion employs a single agent with episodic memory, subsequent work explored explicit role-based specialization across multiple agents.

HuggingGPT [10] employs an LLM coordinator dynamically routing subtasks to specialized expert models, demonstrating that task decomposition and model specialization substantially improve performance on complex workflows. MetaGPT [11] organizes multiple ChatGPT-based agents into a structured software development workflow with specialized roles (Architect, Coder, Tester, Reviewer) operating in a sequential pipeline. MetaGPT's evaluation demonstrated that multi-agent collaboration yields improved code structure, enhanced correctness, and better adherence to engineering best practices compared to single-agent approaches. Similarly, ChatDev [12] simulates a multi-role software team through communicative agents, achieving superior outcomes through collaborative problem-solving.

Dirac Co-Scientist extends these multi-agent paradigms to scientific document generation—a domain requiring different specialization patterns than code synthesis. Our agent

roles (Planning, Analysis, Generation, Verification) parallel software team structures but address distinct challenges: content completeness verification, mathematical consistency checking, and format compliance validation. Unlike prior work focusing primarily on code generation, we incorporate formal correctness guarantees specific to academic writing: preservation of technical accuracy, maintenance of notational consistency, and adherence to publication standards. The shared document graph and knowledge store facilitate agent communication and enable verification agents to directly inform generation refinements—establishing feedback loops beyond simple sequential pipelines.

C. NEURO-SYMBOLIC REASONING AND VERIFICATION

Neuro-symbolic AI combines neural network pattern recognition with symbolic logic's rigor and verifiability [14]. In text generation contexts, neuro-symbolic approaches impose factual and logical constraints on otherwise unconstrained generative models. Prior work in program synthesis has successfully integrated symbolic execution and formal verification to filter incorrect neural generations, creating feedback loops improving reliability [15].

For scientific document generation, ensuring content *faithfulness* is paramount: claims must trace to authoritative sources, mathematical derivations must be valid, and experimental results must be reproducible. Pure neural approaches without grounding have struggled with these requirements, as evidenced by high-profile failures in scientific text generation. Our approach instantiates a neuro-symbolic architecture where an LLM-driven generator produces candidate content, while symbolic modules enforce constraints through citation verification, mathematical consistency checking, and structural validation.

The interplay between neural and symbolic components in Dirac Co-Scientist parallels an expert human editor cross-verifying statements using external references and formal reasoning—a synergy of automated "intuition" and "deduction." This significantly reduces hallucination and increases trustworthiness compared to pure neural generators lacking rigorous oversight. Specific symbolic verification techniques include:

- **Mathematical consistency checking:** Using computer algebra systems (SymPy) to verify equation derivations and formula equivalence between source and generated content.
- **Citation verification:** Cross-referencing all factual claims against retrieved literature and source documents to ensure proper attribution.
- **Structural validation:** Enforcing cross-reference consistency, definition propagation, and notational coherence across document sections.

Recent work on theorem proving with LLMs [16], [17] demonstrates that neural-symbolic integration can achieve formal correctness in mathematical domains. We adapt these principles to document generation, where "correctness"

encompasses factual accuracy, mathematical validity, and structural coherence rather than logical proof.

D. AUTOMATED SCIENTIFIC WRITING AND REPRODUCIBILITY

To our knowledge, no existing system fully automates hierarchical document transformation (paper→thesis→book). However, related efforts highlight AI potential in supporting scientific writing. PaperCoder [19] and Paper2Codes [20] target automated code implementation from research papers, leveraging multi-agent orchestration and retrieval—demonstrating viability of complex pipelines combining LLM reasoning with symbolic components for scientific artifact generation.

Other relevant work includes LLM-assisted literature reviews and technical writing. NotebookLM uses LLMs to help researchers interact with papers and notes. These indicate trends toward AI-assisted research, but lack the systematic approach, formal guarantees, and comprehensive verification mechanisms required for fully automated academic document generation at publication quality.

Dirac Co-Scientist differentiates itself by uniting ideas from these areas into a unified system specifically designed for hierarchical scientific document transformation. It extends RAG to unprecedented scale (multi-stage expansion preserving technical accuracy), employs a multi-agent structure tailored to writing tasks (rather than code synthesis), and integrates neuro-symbolic verification ensuring outputs meet rigorous scientific integrity standards. The system addresses a novel, high-impact problem—automating the months-long process of expanding research papers into comprehensive academic documents—with theoretical guarantees and empirical validation demonstrating production-readiness.

III. MATHEMATICAL FORMULATION AND THEORETICAL GUARANTEES

We formalize Dirac Co-Scientist’s document transformation process as a structured stochastic system with explicit state representations, transformation operators, and convergence properties. This formalization enables rigorous analysis of correctness, complexity, and performance bounds.

A. PRELIMINARIES AND NOTATION

Let Σ^* denote the set of all strings over alphabet Σ , $\mathbb{R}^d$ denote d -dimensional real vector space, $\mathbb{N}$ denote natural numbers, $\mathcal{M}$ denote metadata objects (structured JSON), and $\mathcal{G}$ denote directed labeled graphs. We establish the following core abstractions:

Definition 1 (Document Representation). A document is a tuple $D = \langle \mathcal{S}, \mathcal{M}, \mathcal{R} \rangle$, where:

- $\mathcal{S} = \{s_i\}_{i=1}^n$ is an ordered sequence of content segments with $n = |\mathcal{S}|$
- $\mathcal{M} = (\text{title}, \text{authors}, \text{format}, \text{meta})$ stores global metadata

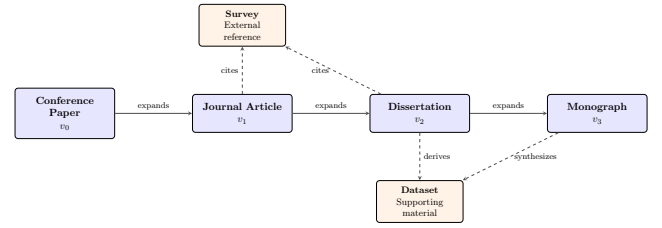


FIGURE 2. Document hierarchy graph structure. Nodes represent documents or document segments. Solid edges labeled “expands” indicate hierarchical expansion relationships (paper→article→thesis→book). Dashed edges labeled “cites” represent citation relationships. The graph maintains acyclicity for expansion relationships while supporting transitive dependency analysis.

- $\mathcal{R} = \{r_j\}_{j=1}^m$ is a set of bibliographic references
- Each segment $s_i = (id_i, \text{type}_i, c_i, \mathbf{e}_i, \mu_i)$ contains:
- $id_i \in \Sigma^*$: unique identifier
 - $\text{type}_i \in \{\text{section}, \text{paragraph}, \text{equation}, \text{figure}, \text{algorithm}\}$: content type (section, paragraph, equation, figure, or algorithm)
 - $c_i \in \Sigma^*$: textual or LaTeX content
 - $\mathbf{e}_i \in \mathbb{R}^d \cup \{\text{None}\}$: optional embedding vector ($d = 1536$ for OpenAI embeddings)
 - $\mu_i \in \mathcal{M}$: segment-specific metadata (position, dependencies, formatting)

The document representation maps to Rust *Document* and *Segment* types, with persistent storage in SurrealDB document and segment tables.

Definition 2 (Document Hierarchy Graph). A document hierarchy is represented by a labeled directed graph $\mathcal{H} = (\mathcal{V}, \mathcal{E}, \lambda)$ where:

- $\mathcal{V} = \{v_i\}_{i=1}^{|\mathcal{V}|}$ is a set of vertices representing documents or document segments
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is a set of directed edges
- $\lambda : \mathcal{E} \rightarrow \{\text{expands}, \text{cites}, \text{derives}, \text{synthesizes}\}$ assigns semantic labels to edges

An edge $(v_i, v_j) \in \mathcal{E}$ with label $\lambda((v_i, v_j)) = \text{expands}$ indicates that document v_j is an expanded version of document v_i . The graph satisfies acyclicity constraints for expansion relationships and maintains transitive closure for dependency analysis.

The hierarchy graph is materialized in SurrealDB as graph relations with typed edges, enabling efficient traversal queries and consistency checking.

Definition 3 (Transformation State). A transformation state at iteration t is a tuple $\mathbf{x}_t = (D^{\text{src}}, D_t^{\text{tgt}}, \mathcal{Q}_t, \mathcal{F}_t, \mathbf{m}_t)$ where:

- D^{src} is the fixed source document (e.g., conference paper)
- $D_t^{\text{tgt}} = \langle \mathcal{S}_t, \mathcal{M}_t, \mathcal{R}_t \rangle$ is the partially generated target document at iteration t
- $\mathcal{Q}_t = \{q_1, \dots, q_{|\mathcal{Q}_t|}\}$ is the task queue of pending generation or correction tasks
- $\mathcal{F}_t = \{f_1, \dots, f_{|\mathcal{F}_t|}\}$ is the feedback history from verification

- $\mathbf{m}_t \in \mathbb{R}^k$ is a vector of performance metrics (completion rate, error counts, token usage)

The initial state $\mathbf{x}_0$ has $D_0^{\text{tgt}} = \langle \emptyset, \mathcal{M}_{\text{init}}, \emptyset \rangle$ (empty target), $\mathcal{Q}_0$ populated from initial planning, and $\mathcal{F}_0 = \emptyset$.

Definition 4 (Content Specification Set). The specification set $\Pi = \{\pi_j\}_{j=1}^m$ is a collection of requirements for the target document, where each $\pi_j = (\phi_j, \text{type}_j, \text{source}_j)$ consists of:

- $\phi_j : D^{\text{tgt}} \times D^{\text{src}} \rightarrow \{\text{true}, \text{false}, \perp\}$: a predicate function mapping target and source documents to satisfaction values ($\perp$ indicates inapplicability)
- $\text{type}_j \in \{\text{completeness}, \text{accuracy}, \text{format}, \text{consistency}\}$: semantic requirement category
- source_j : pointer to source content or external reference justifying the requirement

Specifications are extracted by the planning and analysis agents and refined iteratively. We denote the subset of satisfied specifications at iteration t as $\Pi_{\text{sat}}^{(t)} = \{\pi_j \in \Pi : \phi_j(D_t^{\text{tgt}}, D^{\text{src}}) = \text{true}\}$.

B. STATE TRANSITION DYNAMICS

The transformation process operates through a sequence of state transitions orchestrated by the multi-agent coordination system. Figure 3 illustrates the state transition flow.

Definition 5 (Transformation Operator). The state transition function $\Phi : \mathcal{X} \times \Theta \rightarrow \mathcal{X}$ maps current state and configuration to next state:

$$\mathbf{x}_{t+1} = \Phi(\mathbf{x}_t; \theta) \quad (1)$$

where $\theta \in \Theta$ denotes system configuration parameters including LLM routing policies, retrieval strategies, concurrency limits, and verification thresholds. The transition function is implemented through coordinated agent actions in `coordinator::execute_iteration`.

Each iteration performs the following operations:

- 1) **Task Selection:** Extract ready tasks $\mathcal{B}_t \subseteq \mathcal{Q}_t$ respecting dependency constraints
- 2) **Context Retrieval:** For each task $\tau \in \mathcal{B}_t$, retrieve relevant context $C_\tau = \text{HCDR}(\tau, D^{\text{src}}, D_t^{\text{tgt}}, \mathcal{H})$ using the Hierarchical Contextual Document Retrieval algorithm
- 3) **Content Generation:** Dispatch generation agent to produce content o_τ for task τ given context C_τ
- 4) **Document Update:** Incorporate generated content into target document: $D_{t+1}^{\text{tgt}} = D_t^{\text{tgt}} \oplus \{o_\tau : \tau \in \mathcal{B}_t\}$
- 5) **Verification:** Apply SADV pipeline to check D_{t+1}^{tgt} against specifications Π , producing feedback F_{t+1}
- 6) **Feedback Processing:** Generate correction tasks from feedback and update queue: $\mathcal{Q}_{t+1} = (\mathcal{Q}_t \setminus \mathcal{B}_t) \cup \text{FeedbackTasks}(F_{t+1})$

Definition 6 (Progress Metric). The progress metric $\Psi : D^{\text{tgt}} \times D^{\text{src}} \times \Pi \rightarrow [0, 1]$ quantifies the degree to which

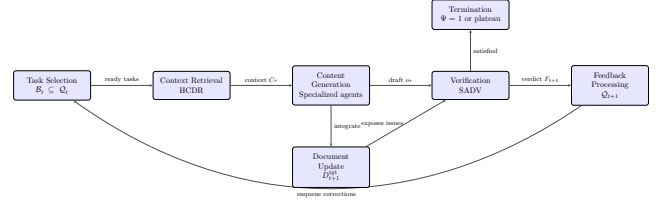


FIGURE 3. State transition dynamics in the document transformation process. The system iteratively transitions from state $\mathbf{x}_t$ to $\mathbf{x}_{t+1}$ through task selection, context retrieval, content generation, verification, and feedback processing. The loop continues until the progress metric Ψ reaches 1 (complete satisfaction) or convergence is detected.

target document D^{tgt} satisfies specification set Π relative to source D^{src} :

$$\Psi(D^{\text{tgt}}, D^{\text{src}}, \Pi) = \frac{|\Pi_{\text{sat}}|}{|\Pi|} = \frac{1}{|\Pi|} \sum_{\pi_j \in \Pi} \mathbb{I}[\phi_j(D^{\text{tgt}}, D^{\text{src}}) = \text{true}] \quad (2)$$

where $\mathbb{I}[\cdot]$ is the indicator function. The metric satisfies $\Psi \in [0, 1]$ with $\Psi = 1$ indicating complete satisfaction of all specifications (successful transformation).

C. CONVERGENCE AND CORRECTNESS GUARANTEES

We now establish formal guarantees for the iterative refinement process, proving convergence to correct outputs under reasonable assumptions.

Theorem 1 (Monotonic Improvement under Sound Verification). Let $\mathcal{H}_t$ denote the history of all states, actions, and observations up to iteration t . Under the following assumptions:

- 1) **Non-zero success probability:** For each generation task τ with context C_τ , there exists $\delta > 0$ such that $\mathbb{P}[\phi_\pi(D^{\text{tgt}}, D^{\text{src}}) = \text{true} \mid D_t^{\text{tgt}}, C_\tau] \geq \delta$ for all $\pi \in \Pi$ related to task τ .
- 2) **Sound verification:** The SADV pipeline is sound, meaning that if $\phi_\pi(D^{\text{tgt}}, D^{\text{src}}) = \text{true}$ for all $\pi \in \Pi$, then verification returns no errors. Furthermore, if $\phi_\pi(D^{\text{tgt}}, D^{\text{src}}) = \text{false}$ for some π , then verification detects it with probability $\geq 1 - \delta_v$ where $\delta_v > 0$ is the verification error rate.
- 3) **Task dependency consistency:** The task queue $\mathcal{Q}_t$ respects the partial order induced by content dependencies, ensuring prerequisite tasks complete before dependent tasks.
- 4) **Bounded concurrency:** The adaptive concurrency controller maintains task parallelism within bounds $[\ell_{\min}, \ell_{\max}]$ for all t .

Then, for all iterations $t \geq 0$:

$$\mathbb{E}[\Psi(D_{t+1}^{\text{tgt}}, D^{\text{src}}, \Pi) \mid \mathcal{H}_t] \geq \Psi(D_t^{\text{tgt}}, D^{\text{src}}, \Pi) \quad (3)$$

with strict inequality when $\Psi(D_t^{\text{tgt}}, D^{\text{src}}, \Pi) < 1$ and $\mathcal{Q}_t \neq \emptyset$. Moreover, the orchestrator terminates almost surely after

at most $T_{\max}$ iterations where $T_{\max} = \mathcal{O}(|\Pi| \cdot (1+r))$ with r being the maximum correction rounds per specification.

Proof. We analyze state transitions through the lens of specification satisfaction. At iteration t , let $\Pi_{\text{unsat}}^{(t)} = \Pi \setminus \Pi_{\text{sat}}^{(t)}$ denote unsatisfied specifications. For each ready task $\tau \in \mathcal{B}_t$, the coordinator:

- 1) Retrieves context C_τ containing relevant source material and external references
- 2) Invokes generation agent producing content o_τ addressing one or more unsatisfied specifications
- 3) Updates target document D_t^{tgt} by incorporating o_τ

By assumption (1), each generation attempt has non-zero probability $\delta > 0$ of satisfying at least one previously unsatisfied specification. Task execution either: (i) adds content satisfying new specifications without violating existing ones, or (ii) corrects violations identified by verification. In case (i), $\Psi(D_{t+1}^{\text{tgt}}, D^{\text{src}}, \Pi) \geq \Psi(D_t^{\text{tgt}}, D^{\text{src}}, \Pi)$ with probability $\geq \delta$ of strict improvement. In case (ii), correction tasks target specific violations, and by assumption (1) and iterative refinement, $\mathbb{E}[\Psi(D_{t+1}^{\text{tgt}}, D^{\text{src}}, \Pi)] \geq \Psi(D_t^{\text{tgt}}, D^{\text{src}}, \Pi)$.

By assumption (2), sound verification ensures detected errors trigger corrective actions, preventing regression. Assumption (3) ensures logical task ordering prevents circular dependencies causing non-monotonic behavior. Assumption (4) guarantees progress is not artificially constrained by concurrency bottlenecks.

For termination, the implementation enforces three stopping conditions:

- 1) Perfect satisfaction: $\Psi(D_t^{\text{tgt}}, D^{\text{src}}, \Pi) = 1$
- 2) Convergence detection: Ψ unchanged for k_{plateau} consecutive iterations
- 3) Iteration cap: $t \geq T_{\max}$

Under assumptions (1)-(4), condition (1) occurs with probability approaching 1 as $t \rightarrow \infty$ by non-zero success probability and iterative refinement. The bound $T_{\max} = \mathcal{O}(|\Pi| \cdot (1+r))$ follows from at most $|\Pi|$ initial generation tasks plus $r \cdot |\Pi|$ correction tasks, with each iteration processing ≥ 1 task when queue is non-empty. $\square$ $\square$

Corollary 2 (Iteration and Computational Complexity). *Under the assumptions of Theorem 1, let $|\Pi|$ denote the number of specifications and $r \geq 1$ the maximum correction rounds per specification. Then:*

- 1) **Task complexity:** The system executes at most $T_{\text{task}} = (1+r)|\Pi|$ generation tasks and $T_{\text{verify}} = r|\Pi|$ verification rounds, yielding total task count $\mathcal{O}(r \cdot |\Pi|)$.
- 2) **Wall-clock time:** Execution time is bounded by $\mathcal{O}\left(\frac{r \cdot |\Pi|}{\ell_{\min}} \cdot T_{\text{LLM}} + r \cdot |\Pi| \cdot T_{\text{verify}}\right)$, where T_{LLM} is average LLM inference latency and T_{verify} is verification time per task.
- 3) **LLM call complexity:** The number of LLM inference calls is $\mathcal{O}(r \cdot |\Pi|)$, with each task requiring one primary inference plus potential retries under rate limiting.

We now establish bounds on hallucination probability as a function of retrieval quality.

Theorem 3 (Faithfulness and Hallucination Bound). *Let $H_0 \in [0, 1]$ denote the baseline hallucination rate of a vanilla LLM generating document D^{tgt} from source D^{src} without retrieval augmentation. Let $k \geq 1$ be the number of distinct contexts retrieved per generation task via HCDR, and let $\rho \in [0, 1]$ be the probability that a randomly selected retrieved context contains information sufficient for correctly addressing the task. Under the following assumptions:*

- 1) **Context independence:** Utility of retrieved contexts $\{C_1, \dots, C_k\}$ are mutually independent Bernoulli random variables
- 2) **Successful grounding eliminates hallucination:** When at least one context C_i provides correct information, the agent uses it faithfully, resulting in zero hallucination for that content
- 3) **Content modularity:** Hallucination events across different generation tasks are independent

Then the expected hallucination rate for the final document is bounded by:

$$\mathbb{E}[H(D_{\text{final}}^{\text{tgt}})] \leq (1-\rho)^k \cdot H_0 + \epsilon_{\text{multi-agent}} \quad (4)$$

where $\epsilon_{\text{multi-agent}} \geq 0$ accounts for residual hallucination due to coordination errors, specification mismatches, or incomplete verification coverage, and $H(D)$ denotes the proportion of content in document D that cannot be traced to source material or verified references.

Proof. For a single generation task τ with retrieved contexts $C_\tau = \{C_1, \dots, C_k\}$, define indicator random variables $X_i \sim \text{Bernoulli}(\rho)$ where $X_i = 1$ indicates context C_i contains sufficient correct information for task τ .

The agent hallucinates only if all contexts fail to provide grounding: $\bigcap_{i=1}^k \{X_i = 0\}$. Under assumption (1):

$$\mathbb{P}\left(\bigcap_{i=1}^k \{X_i = 0\}\right) = \prod_{i=1}^k \mathbb{P}(X_i = 0) = (1-\rho)^k \quad (5)$$

When all contexts fail, the agent reverts to parametric knowledge with baseline hallucination rate H_0 . When at least one context succeeds, assumption (2) implies faithful generation with zero hallucination for that task. Multi-agent coordination introduces residual errors $\epsilon_{\text{multi-agent}}$ from planning inaccuracies, misinterpreted specifications, or undetected verification failures.

For a single task τ , expected hallucination rate is:

$$\mathbb{E}[H_{\text{task}}(\tau)] = (1-\rho)^k \cdot H_0 + \epsilon_{\text{multi-agent}} \quad (6)$$

For document $D_{\text{final}}^{\text{tgt}}$ comprising m generation tasks $\{\tau_1, \dots, \tau_m\}$, assumption (3) implies overall hallucination is the average:

$$\mathbb{E}[H(D_{\text{final}}^{\text{tgt}})] = \frac{1}{m} \sum_{j=1}^m \mathbb{E}[H_{\text{task}}(\tau_j)] \leq (1-\rho)^k \cdot H_0 + \epsilon_{\text{multi-agent}} \quad (7)$$

The bound demonstrates exponential reduction in hallucination as k increases or ρ improves, providing theoretical justification for multi-context retrieval. $\square$ $\square$

Proposition 4 (Retrieval Quality and Fidelity). *Let $\mathcal{I}(D^{src})$ denote the information content of source document D^{src} and $\mathcal{I}_{ret}(C)$ the information content of retrieved context set C . Define retrieval coverage as $\gamma = \frac{|\mathcal{I}_{ret}(C) \cap \mathcal{I}(D^{src})|}{|\mathcal{I}(D^{src})|}$. Then content fidelity $F(D^{tgt}, D^{src}) \in [0, 1]$ measuring information preservation satisfies:*

$$F(D^{tgt}, D^{src}) \geq \gamma \cdot (1 - H(D^{tgt})) - \epsilon_{loss} \quad (8)$$

where $\epsilon_{loss} \geq 0$ accounts for information loss during generation (summarization, paraphrasing, compression).

Proof. Content fidelity measures the proportion of source information preserved in the target document. The retrieved context set C contains information $\mathcal{I}_{ret}(C)$, with coverage γ representing the fraction of source information included in retrieved contexts.

The generation process transforms retrieved context into target content. Let $\mathcal{I}(D^{tgt})$ denote the information content of the target document. The preservation process is subject to:

- Information loss ϵ_{loss} from summarization and paraphrasing, reducing information density
- Hallucination rate $H(D^{tgt})$, introducing non-source content that does not contribute to fidelity

The information preserved from retrieval is at most $\gamma \cdot |\mathcal{I}(D^{src})|$. Accounting for hallucination, only $(1 - H(D^{tgt}))$ fraction of target content is faithful to source. Information loss during generation further reduces preserved content by ϵ_{loss} . Therefore:

$$F(D^{tgt}, D^{src}) = \frac{|\mathcal{I}(D^{tgt}) \cap \mathcal{I}(D^{src})|}{|\mathcal{I}(D^{src})|} \geq \gamma \cdot (1 - H(D^{tgt})) - \epsilon_{loss}$$

This establishes the lower bound. $\square$ $\square$

This proposition formalizes the intuition that high retrieval coverage γ and low hallucination H jointly ensure high fidelity transformation, providing theoretical justification for our hybrid retrieval approach.

D. DOCUMENT GRAPH PROPERTIES

The document hierarchy graph $\mathcal{H}$ satisfies several structural properties enabling efficient reasoning.

Lemma 5 (Hierarchy Acyclicity). *The subgraph $\mathcal{H}_{exp} \subseteq \mathcal{H}$ induced by edges with label $\lambda(e) = \text{expands}$ is a directed acyclic graph (DAG). This ensures well-defined document hierarchy levels without circular expansion relationships.*

Proof. By construction, expansion edges are directed from concise to comprehensive documents following the natural hierarchy paper $\rightarrow$ article $\rightarrow$ thesis $\rightarrow$ book. Cycles would imply a document expands itself, contradicting the definition of hierarchical expansion. The DAG structure enables topological sorting for dependency-aware task scheduling. $\square$ $\square$

Lemma 6 (Transitive Dependency Closure). *For any path $v_1 \xrightarrow{\text{expands}} v_2 \xrightarrow{\text{expands}} \dots \xrightarrow{\text{expands}} v_k$ in $\mathcal{H}$, the content of v_k transitively depends on all content in $\{v_1, \dots, v_{k-1}\}$. The system maintains transitive closure $\mathcal{H}^*$ enabling efficient ancestor queries.*

This property informs retrieval strategy: when generating content for v_k , the system retrieves relevant segments from all ancestors, ensuring content completeness.

IV. SYSTEM ARCHITECTURE

Dirac Co-Scientist implements a layered architecture integrating document processing, multi-agent orchestration, knowledge management, and verification. Figure 4 presents the overall system design.

A. ARCHITECTURAL PRINCIPLES

The system adheres to several design principles:

- **Modularity:** Trait-based abstractions (Agent, DocumentProcessor, Retriever, Verifier) enable pluggable implementations and independent component testing.
- **Hierarchical Decomposition:** Document transformation is decomposed into planning, analysis, generation, and verification phases, each handled by specialized agents with domain-specific capabilities.
- **Persistent Knowledge Fabric:** SurrealDB provides unified storage for documents, embeddings, hierarchy graphs, and verification artifacts, enabling cross-session consistency and knowledge reuse.
- **Hybrid Retrieval:** Combining vector similarity search with graph-based reasoning ensures high precision and recall in context retrieval, critical for minimizing hallucinations.
- **Iterative Refinement:** Verification-driven feedback loops with formal guarantees ensure convergence to correct outputs through systematic error correction.
- **Observability:** Comprehensive logging, metrics collection, and real-time monitoring enable debugging, performance optimization, and quality assurance.

B. DOCUMENT PROCESSING LAYER

The document processing layer handles ingestion, parsing, and structural analysis of input documents.

1) PDF Parser and Content Extractor

The `DocumentParser` module (`src/document/parser.rs`) processes input PDFs through a multi-stage extraction pipeline:

- 1) **Text extraction:** Using PDF.js or PyPDF2, extract raw text with position information preserving reading order
- 2) **Structure identification:** Detect document sections, subsections, paragraphs, equations, figures, tables, and captions using heuristic rules and layout analysis

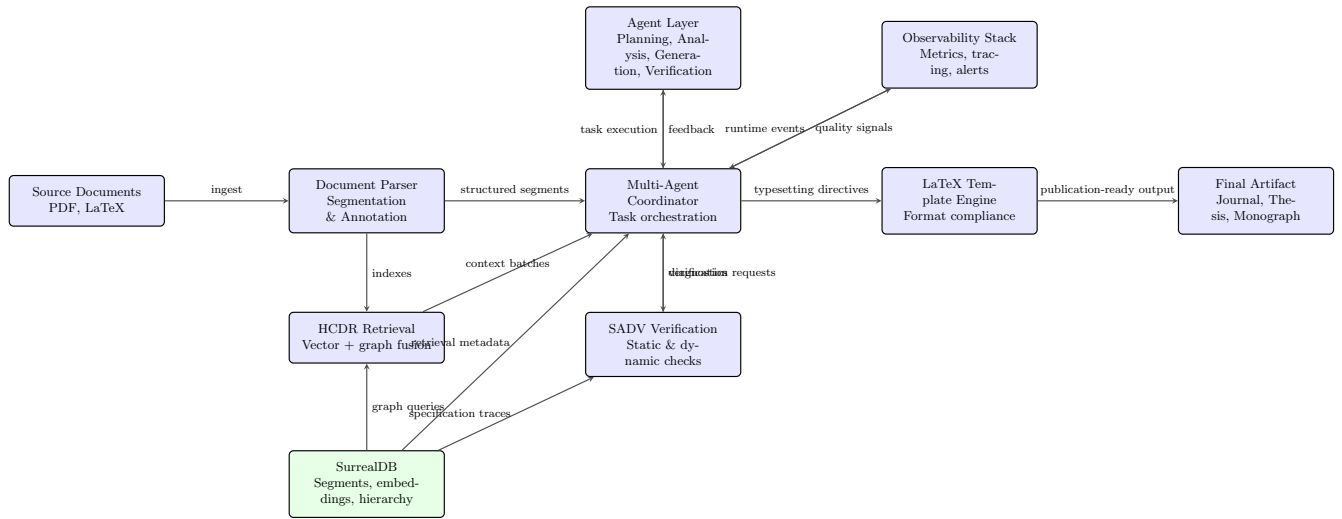


FIGURE 4. Dirac Co-Scientist system architecture. The document parser extracts structured content from input PDFs. The multi-agent coordinator orchestrates specialized agents (Planning, Analysis, Generation, Verification) through iterative refinement cycles. A unified SurrealDB knowledge store maintains document segments, embeddings, and hierarchy graphs. The HCDR engine provides hybrid vector-graph retrieval, while the SADV pipeline enforces formal verification. The LaTeX Template Engine ensures format compliance in generated outputs.

- 3) **LaTeX reconstruction:** For papers with source available, parse LaTeX directly; otherwise, infer LaTeX from PDF rendering (extracting equations via Mathpix or similar OCR)
- 4) **Segmentation:** Split document into semantic segments (definition, theorem, example, etc.) based on structural cues
- 5) **Embedding generation:** Compute vector embeddings for each segment using SciBERT or similar domain-specific models
- 6) **Metadata extraction:** Extract bibliographic information, author affiliations, publication venue, keywords, and citation graph

The extracted segments and metadata are persisted to SurrealDB with unique identifiers enabling cross-referencing.

2) Domain and Format Analyzer

The `FormatAnalyzer` module identifies document characteristics informing transformation strategy:

- **Domain classification:** Using title, abstract, and keywords, classify document into scientific domain (computer science, physics, biology, mathematics) and sub-domain (e.g., machine learning, quantum mechanics)
- **Format identification:** Detect source format (conference paper, journal article, thesis chapter) based on structural patterns, length, and citation styles
- **Complexity estimation:** Assess document complexity based on mathematical density, citation count, figure/table prevalence, and technical terminology concentration

This analysis informs agent configuration, retrieval strategy selection, and quality threshold calibration.

TABLE 2. Dirac Co-Scientist Agent Specialization and Responsibilities

Agent	Responsibilities and Capabilities
Planning	Analyzes source document structure; generates hierarchical outline for target document; identifies content gaps requiring expansion; estimates section lengths and complexity; produces dependency-ordered task graph
Analysis	Extracts key concepts, definitions, theorems from source; identifies relationships between technical components; suggests background material requiring elaboration; produces refined specifications for generation tasks
Generation	Synthesizes content for target sections using retrieved context; maintains consistent notation and terminology; generates LaTeX-formatted prose with proper citations; adapts writing style to target format conventions
Verification	Validates content completeness against source; checks mathematical consistency of equations; verifies citation accuracy and reference completeness; enforces format compliance; detects hallucinations through fact-checking

C. MULTI-AGENT ORCHESTRATION LAYER

The orchestration layer coordinates specialized agents through structured workflows defined by the transformation state machine.

1) Agent Specialization

Table 2 summarizes agent roles and responsibilities.

Each agent is implemented as a specialized LLM prompt template with carefully engineered system messages, few-shot examples, and output format specifications. The coordinator selects appropriate LLM backends for each agent

based on task requirements (long-context models for planning, fast inference models for verification, specialized scientific models for generation).

2) Coordination Protocol

The `Coordinator` (`src/coordinator/mod.rs`) implements the orchestration algorithm (Algorithm 2):

- 1) **Initialization:** Load source document, initialize document hierarchy graph, configure target format specifications
- 2) **Planning Phase:** Invoke Planning Agent to generate transformation outline and task dependency graph
- 3) **Task Queue Management:** Maintain priority queue of pending tasks with dependency constraints; select ready tasks for parallel execution
- 4) **Agent Dispatch:** For each ready task, retrieve context via HCDR, invoke appropriate agent with context-augmented prompt, collect generated output
- 5) **Document Integration:** Merge generated content into target document, updating hierarchy graph and metadata
- 6) **Verification Phase:** Apply SADV pipeline to validate generated content; collect feedback identifying errors or omissions
- 7) **Feedback Processing:** Generate correction tasks from verification feedback; prioritize high-severity issues; update task queue
- 8) **Convergence Check:** Monitor progress metric Ψ ; terminate when target met or plateau detected

The coordinator maintains comprehensive state including document versions, task history, verification reports, and performance metrics, enabling debugging and iterative improvement.

D. KNOWLEDGE MANAGEMENT LAYER

The knowledge layer provides persistent storage, indexing, and retrieval capabilities supporting all system components.

1) SurrealDB Integration

SurrealDB serves as a unified multi-model database supporting:

- **Document storage:** Documents and segments stored as typed records with schema validation
- **Vector index:** HNSW index on segment embeddings enabling efficient semantic search
- **Graph database:** Document hierarchy graph with typed edges supporting graph queries
- **Full-text search:** BM25 index on segment content for lexical matching
- **Metadata store:** Flexible JSON storage for verification artifacts, task logs, and metrics

The `StorageManager` (`src/storage/manager.rs`) provides typed interfaces for SurrealDB operations, handling connection pooling, transactions, and schema migrations.

2) Document Graph Reasoner

The `DocGraphReasoner` module analyzes the document hierarchy graph $\mathcal{H}$ to support intelligent retrieval and consistency checking:

- **Ancestor retrieval:** Given target document section, identify all source sections it transitively depends on
- **Consistency verification:** Check that definitions, notation, and assumptions introduced in source documents are properly propagated to dependent documents
- **Content reuse analysis:** Identify opportunities for reusing generated content across multiple target documents (e.g., shared background sections)
- **Dependency impact analysis:** When source content changes, identify all dependent target sections requiring regeneration

The reasoner uses graph traversal algorithms (BFS, topological sort) optimized with caching for frequent queries.

E. VERIFICATION AND QUALITY ASSURANCE LAYER

The verification layer ensures generated content meets quality, correctness, and format requirements.

1) Symbolically-Augmented Document Verification (SADV)

The SADV pipeline (`src/verification/sadv.rs`) implements multi-layered verification:

1) Structural Validation

- Section completeness: verify all planned sections generated
- Cross-reference consistency: check all figure/table/equation references resolve
- Citation completeness: ensure all cited works appear in bibliography
- Notation consistency: verify mathematical symbols used consistently

2) Content Verification

- Source fidelity: check all key content from source preserved in target
- Fact-checking: verify factual claims against retrieved references
- Mathematical consistency: use SymPy to validate equation derivations and transformations
- Plagiarism detection: ensure generated content sufficiently paraphrased and attributed

3) Format Compliance

- LaTeX syntax validation: parse generated LaTeX checking for errors
- Template conformance: verify document structure matches target template requirements
- Style consistency: check citation format, heading styles, equation numbering conventions

For each detected issue, SADV generates structured feedback including issue type, severity, location, and suggested correction, enabling targeted agent-driven refinement.

2) Mathematical Verification Tools

For documents with significant mathematical content, SADV integrates symbolic reasoning tools:

- **SymPy:** Computer algebra system verifying equation equivalence, performing symbolic differentiation/integration, simplifying expressions
- **Z3 SMT Solver:** Checking satisfiability of logical constraints, verifying inequality relationships, validating complexity bounds
- **Theorem provers:** For documents with formal proofs, integrating with Lean or Coq for mechanized verification (experimental)

These tools operate on extracted mathematical expressions, comparing source and target formulas to detect inconsistencies or errors introduced during generation.

F. TEMPLATE AND FORMAT MANAGEMENT

The template engine ensures generated documents conform to publication standards.

1) LaTeX Template Inference Engine

The `TemplateEngine` module (`src/template/engine.rs`) manages format-specific document generation:

- **Template library:** Maintains parameterized LaTeX templates for common formats (IEEE transactions, ACM proceedings, Springer LNCS, university thesis templates)
- **Style inference:** Analyzes source document to infer appropriate target style (single/double column, citation format, font specifications)
- **Macro synthesis:** Generates custom LaTeX macros ensuring notation consistency and simplifying agent prompts
- **Structure generation:** Automatically produces front matter (title page, abstract, acknowledgments, table of contents), back matter (bibliography, appendices, index)
- **Format validation:** Compiles generated LaTeX with appropriate toolchain (pdflatex, xelatex) verifying successful compilation

The template engine operates as a post-processor on generated content, ensuring format compliance without burdening generation agents with formatting details.

V. CORE ALGORITHMS AND METHODS

We present the key algorithms underpinning Dirac Co-Scientist: hierarchical contextual document retrieval (HCDR), multi-agent orchestration, and symbolically-augmented document verification (SADV).

A. HIERARCHICAL CONTEXTUAL DOCUMENT RETRIEVAL (HCDR)

The HCDR algorithm (Algorithm 1) combines semantic vector search, lexical BM25 matching, and document graph

Algorithm 1 Hierarchical Contextual Document Retrieval (HCDR)

Input: Task τ (section to generate), source document D^{src} , current target D_t^{tgt} , hierarchy graph $\mathcal{H}$, external corpus $\mathcal{E}$
Input: Parameters: $k_{\text{sem}}, k_{\text{lex}}, k_{\text{ext}}, \alpha, \beta, \gamma$
Output: Context set C of relevant segments

```

1: query  $\leftarrow$  EXTRACTKEYWORDS( $\tau$ )  $\triangleright$  Extract task description keywords
2:  $e_{\text{query}} \leftarrow$  EMBED(query)  $\triangleright$  Compute query embedding
3:
4: // Phase 1: Source document retrieval
5:  $C_{\text{src}} \leftarrow \emptyset$ 
6: for all segment  $s_i \in D^{\text{src}}.S$  do
7:    $\text{score}_{\text{sem}} \leftarrow \cos(e_{\text{query}}, s_i.e)$   $\triangleright$  Semantic similarity
8:    $\text{score}_{\text{lex}} \leftarrow \text{BM25}(\text{query}, s_i.c)$   $\triangleright$  Lexical matching
9:    $\text{score}_i \leftarrow \alpha \cdot \text{score}_{\text{sem}} + \beta \cdot \text{score}_{\text{lex}}$ 
10: end for
11:  $C_{\text{src}} \leftarrow \text{TOPK}(\{(s_i, \text{score}_i)\}, k_{\text{sem}})$   $\triangleright$  Select top- $k_{\text{sem}}$  segments
12:
13: // Phase 2: Hierarchical graph-based expansion
14:  $C_{\text{hier}} \leftarrow \emptyset$ 
15:  $v_{\tau} \leftarrow \text{IDENTIFYGRAPHNODE}(\tau, \mathcal{H})$   $\triangleright$  Locate task in hierarchy
16: if  $v_{\tau} \neq \text{None}$  then
17:    $\mathcal{A} \leftarrow \text{ANCESTORS}(v_{\tau}, \mathcal{H})$   $\triangleright$  Get all ancestor nodes
18:   for all ancestor  $a \in \mathcal{A}$  do
19:      $C_{\text{hier}} \leftarrow C_{\text{hier}} \cup \text{GETSEGMENTS}(a)$   $\triangleright$  Include ancestor content
20:   end for
21: end if
22:
23: // Phase 3: External reference retrieval
24:  $C_{\text{ext}} \leftarrow \emptyset$ 
25:  $\mathcal{R}_{\text{cited}} \leftarrow \text{EXTRACTCITATIONS}(C_{\text{src}} \cup C_{\text{hier}})$   $\triangleright$  Get cited references
26: for all reference  $r \in \mathcal{R}_{\text{cited}}$  do
27:   if  $r \in \mathcal{E}$  then
28:      $C_{\text{ext}} \leftarrow C_{\text{ext}} \cup \text{RETRIEVESUMMARY}(r, \mathcal{E})$ 
29:   end if
30: end for
31:  $C_{\text{ext}} \leftarrow C_{\text{ext}} \cup \text{SEMANTICSEARCH}(\text{query}, \mathcal{E}, k_{\text{ext}})$   $\triangleright$  General semantic search
32:
33: // Phase 4: Target document context
34:  $C_{\text{tgt}} \leftarrow \text{GETPRECEDINGSECTIONS}(D_t^{\text{tgt}}, \tau)$   $\triangleright$  Include already-generated sections
35:
36: // Phase 5: Context aggregation and ranking
37:  $C_{\text{all}} \leftarrow C_{\text{src}} \cup C_{\text{hier}} \cup C_{\text{ext}} \cup C_{\text{tgt}}$ 
38:  $C_{\text{all}} \leftarrow \text{DEDUPLICATE}(C_{\text{all}})$   $\triangleright$  Remove duplicate segments
39:  $C_{\text{all}} \leftarrow \text{RERANKBYRELEVANCE}(C_{\text{all}}, \text{query})$   $\triangleright$  Final ranking
40:  $C \leftarrow \text{LIMITTOKENS}(C_{\text{all}}, T_{\text{max}})$   $\triangleright$  Respect context window limit
41:
42: return  $C$   $\triangleright$  Return ranked, filtered context

```

traversal to retrieve comprehensive, relevant context for generation tasks.

1) Algorithm Description

HCDR operates in five phases, as illustrated in Figure 5:

Phase 1: Source Document Retrieval computes relevance scores combining semantic similarity (cosine distance between query and segment embeddings) and lexical overlap (BM25). This hybrid scoring ensures both conceptually related content and technically precise terminology are retrieved.

Phase 2: Hierarchical Graph Expansion leverages the document hierarchy graph $\mathcal{H}$ to identify ancestor documents and sections. When generating thesis chapter content,

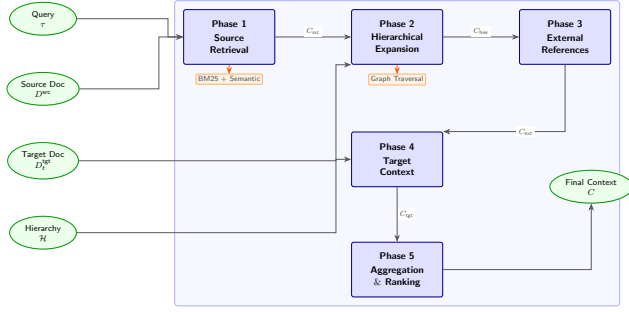


FIGURE 5. HCDR algorithm flow. The algorithm processes a query task τ through five phases: (1) semantic and lexical retrieval from source document, (2) hierarchical graph expansion to include ancestor content, (3) external reference retrieval, (4) target document context inclusion, and (5) aggregation and ranking with token limit enforcement.

this phase ensures relevant paper and article sections are included, maintaining content continuity across hierarchy levels.

Phase 3: External Reference Retrieval extracts citations from already-retrieved context and fetches cited documents from the external corpus $\mathcal{E}$. This grounds generated content in established literature, reducing hallucinations and ensuring proper attribution.

Phase 4: Target Document Context includes previously generated sections from the current target document, ensuring notational consistency, definition reuse, and logical flow.

Phase 5: Aggregation and Ranking deduplicates segments, re-ranks by final relevance scores, and limits total context to respect LLM token windows (typically $T_{\max} = 16384$ or 32768 tokens).

2) Complexity Analysis

Theorem 7 (HCDR Computational Complexity). *The HCDR algorithm has time complexity $\mathcal{O}(|\mathcal{S}| \cdot d + |\mathcal{S}| \log |\mathcal{S}| + |\mathcal{A}| + |\mathcal{R}|)$, where $|\mathcal{S}|$ is the number of source segments, d is embedding dimension, $|\mathcal{A}|$ is the number of ancestors in hierarchy graph, and $|\mathcal{R}|$ is the number of cited references. With HNSW indexing, vector similarity queries reduce to $\mathcal{O}(\log |\mathcal{S}|)$ per query.*

Proof. Phase 1 semantic scoring requires $|\mathcal{S}|$ cosine similarity computations at $\mathcal{O}(d)$ each, totaling $\mathcal{O}(|\mathcal{S}| \cdot d)$. BM25 scoring is $\mathcal{O}(|\mathcal{S}| \cdot |Q|)$ where $|Q|$ is query length (typically $|Q| \ll |\mathcal{S}|$). Top- k selection requires sorting: $\mathcal{O}(|\mathcal{S}| \log |\mathcal{S}|)$. With HNSW indexing, Phase 1 reduces to $\mathcal{O}(k_{\text{sem}} \log |\mathcal{S}|)$.

Phase 2 graph traversal uses breadth-first search finding ancestors in $\mathcal{O}(|\mathcal{A}|)$. Phase 3 reference retrieval requires $|\mathcal{R}|$ lookups at $\mathcal{O}(1)$ each with hash indexing. Phase 4 is $\mathcal{O}(|\mathcal{S}_{\text{tgt}}|)$ where $|\mathcal{S}_{\text{tgt}}| \ll |\mathcal{S}|$. Phase 5 deduplication uses hash sets: $\mathcal{O}(|\mathcal{C}_{\text{all}}|)$. Overall complexity is dominated by vector search and sorting. $\square$ $\square$

Empirically, with HNSW indexing and typical parameter settings ($k_{\text{sem}} = 10$, $|\mathcal{A}| \approx 5$, $|\mathcal{R}| \approx 15$), HCDR executes in 50-200ms, negligible compared to LLM inference latency (5-30 seconds).

Algorithm 2 Multi-Agent Document Transformation Orchestration

Input: Source document D^{src} , target format specification F , specification set Π

Input: Configuration θ : LLM routing, concurrency limits, convergence thresholds

Output: Generated document $D_{\text{final}}^{\text{tgt}}$

```

1: // Initialization
2:  $\mathcal{H} \leftarrow \text{BUILDHIERARCHYGRAPH}(D^{\text{src}}, F)$   $\triangleright$  Construct document graph
3:  $M \leftarrow \text{PLANNINGAGENT}(D^{\text{src}}, F)$   $\triangleright$  Generate transformation outline
4:  $\mathcal{Q}_0 \leftarrow \text{TASKGRAPH}(M)$   $\triangleright$  Initialize task queue from outline
5:  $D_0^{\text{tgt}} \leftarrow \langle \emptyset, \mathcal{M}_{\text{init}}, \emptyset \rangle$   $\triangleright$  Empty target document
6:  $\mathcal{F}_0 \leftarrow \emptyset; t \leftarrow 0$ 
7:
8: // Main transformation loop
9: while  $\Psi(D_t^{\text{tgt}}, D^{\text{src}}, \Pi) < 1$  and  $t < T_{\max}$  do
10:  $\mathcal{B}_t \leftarrow \text{READYTASKS}(\mathcal{Q}_t, D_t^{\text{tgt}})$   $\triangleright$  Select tasks with satisfied dependencies
11:
12: // Parallel task execution with adaptive concurrency
13: for all  $\tau \in \mathcal{B}_t$  in parallel (limit  $\ell_t$ ) do
14:   if  $\tau.\text{type} = \text{generation}$  then
15:      $C_\tau \leftarrow \text{HCDR}(\tau, D^{\text{src}}, D_t^{\text{tgt}}, \mathcal{H})$   $\triangleright$  Retrieve context
16:      $o_\tau \leftarrow \text{GENERATIONAGENT}(\tau, C_\tau, F)$   $\triangleright$  Generate content
17:      $D_t^{\text{tgt}} \leftarrow D_t^{\text{tgt}} \oplus o_\tau$   $\triangleright$  Integrate into document
18:   else if  $\tau.\text{type} = \text{correction}$  then
19:      $f \leftarrow \tau.\text{feedback}$   $\triangleright$  Get verification feedback
20:      $C_\tau \leftarrow \text{HCDR}(f, D^{\text{src}}, D_t^{\text{tgt}}, \mathcal{H})$   $\triangleright$  Context for correction
21:      $o_\tau \leftarrow \text{GENERATIONAGENT}(\tau, C_\tau, f)$   $\triangleright$  Generate correction
22:      $D_t^{\text{tgt}} \leftarrow \text{REPLACE}(D_t^{\text{tgt}}, \tau.\text{section}, o_\tau)$   $\triangleright$  Apply correction
23:   end if
24:    $\mathcal{Q}_t \leftarrow \mathcal{Q}_t \setminus \{\tau\}$   $\triangleright$  Mark task complete
25: end for
26:
27: // Verification and feedback generation
28:  $F_{t+1} \leftarrow \text{SADVPIPELINE}(D_t^{\text{tgt}}, D^{\text{src}}, \Pi)$   $\triangleright$  Verify generated content
29: if  $F_{t+1} \neq \emptyset$  then
30:   for all issue  $f \in F_{t+1}$  do
31:      $\tau_f \leftarrow \text{CREATECORRECTIONTASK}(f)$   $\triangleright$  Generate correction task
32:      $\mathcal{Q}_t \leftarrow \mathcal{Q}_t \cup \{\tau_f\}$   $\triangleright$  Add to task queue
33:   end for
34: end if
35:
36: // Adaptive concurrency adjustment
37:  $p_t \leftarrow \text{SUCCESSRATE}(\mathcal{B}_t)$   $\triangleright$  Compute task success rate
38:  $\ell_{t+1} \leftarrow \text{ADAPTCONCURRENCY}(\ell_t, p_t)$   $\triangleright$  Adjust parallelism
39:
40: // Convergence monitoring
41:  $\mathcal{F}_t \leftarrow \mathcal{F}_t \cup F_{t+1}$ 
42: if  $\text{DETECTPLATEAU}(\Psi, t, k_{\text{plateau}})$  then
43:   break  $\triangleright$  Stop if progress plateaus
44: end if
45:  $t \leftarrow t + 1$ 
46: end while
47:
48: // Finalization
49:  $D_{\text{final}}^{\text{tgt}} \leftarrow \text{TEMPLATEENGINE}(D_t^{\text{tgt}}, F)$   $\triangleright$  Apply format template
50:  $\text{STOREPERSISTENT}(D_{\text{final}}^{\text{tgt}}, \mathcal{H}, \mathcal{F}_t)$   $\triangleright$  Save to SurrealDB
51: return  $D_{\text{final}}^{\text{tgt}}$ 

```

B. MULTI-AGENT ORCHESTRATION ALGORITHM

Algorithm 2 presents the complete orchestration workflow.

1) Orchestration Details

The orchestration workflow is visualized in Figure 6.

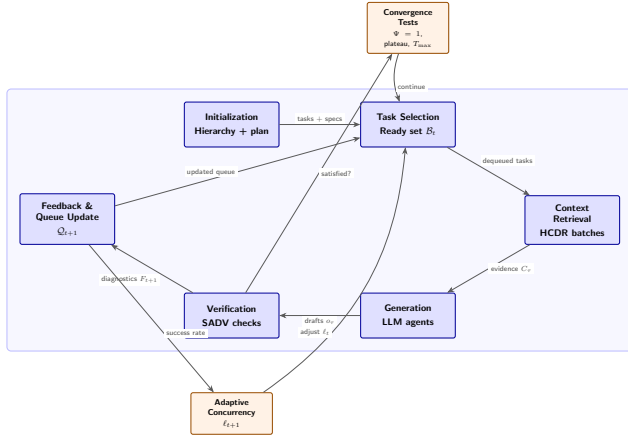


FIGURE 6. Multi-agent orchestration workflow. The system initializes with hierarchy graph construction and planning, then enters an iterative loop of task selection, context retrieval, content generation, verification, and feedback processing. The loop continues until convergence ($\Psi = 1$) or maximum iterations reached.

Initialization constructs the document hierarchy graph encoding relationships between source and target documents, invokes the Planning Agent to generate a transformation outline, and populates the task queue with generation tasks ordered by dependencies.

Main Loop iteratively selects ready tasks (those with satisfied dependencies), executes them in parallel subject to concurrency limits, integrates generated content, and verifies completeness and correctness. Verification feedback generates correction tasks added to the queue, creating an iterative refinement cycle.

Adaptive Concurrency monitors task success rates and dynamically adjusts parallelism. High success rates enable increased concurrency; frequent failures (e.g., rate limiting, generation errors) trigger backoff to lower concurrency, improving stability.

Convergence Detection monitors the progress metric Ψ . The loop terminates when $\Psi = 1$ (complete satisfaction), progress plateaus (unchanged for k_{plateau} iterations), or the maximum iteration count T_{max} is reached.

Finalization applies the LaTeX Template Engine to ensure format compliance, compiles the document to verify correctness, and persists all artifacts to SurrealDB for future reference.

C. SYMBOLICALLY-AUGMENTED DOCUMENT VERIFICATION (SADV)

Algorithm 3 details the verification pipeline.

1) Verification Details

Phase 1: Structural Validation ensures the document contains all required sections, all cross-references (figures, tables, equations, sections) resolve correctly, and all citations appear in the bibliography. These checks prevent common structural errors.

Algorithm 3 Symbolically-Augmented Document Verification (SADV)

Input: Target document D^{tgt} , source document D^{src} , specifications Π

Output: Feedback set F (empty if no issues)

```

1:  $F \leftarrow \emptyset$  ▷ Initialize feedback list
2:
3: // Phase 1: Structural validation
4:  $\text{sections}_{\text{exp}} \leftarrow \Pi_{\text{structure}}$  ▷ Expected sections from specifications
5:  $\text{sections}_{\text{act}} \leftarrow \text{EXTRACTSECTIONS}(D^{\text{tgt}})$  ▷ Actual sections in document
6: if  $\text{sections}_{\text{act}} \not\supseteq \text{sections}_{\text{exp}}$  then
7:    $F \leftarrow F \cup \{\text{"Missing sections: " + } (\text{sections}_{\text{exp}} \setminus \text{sections}_{\text{act}})\}$ 
8: end if
9:
10:  $\text{refs} \leftarrow \text{EXTRACTREFERENCES}(D^{\text{tgt}})$  ▷ Extract all cross-references
11: for all  $\text{ref} \in \text{refs}$  do
12:   if  $\neg \text{RESOLVES}(\text{ref}, D^{\text{tgt}})$  then
13:      $F \leftarrow F \cup \{\text{"Unresolved reference: " + ref}\}$ 
14:   end if
15: end for
16:
17:  $\text{cites} \leftarrow \text{EXTRACTCITATIONS}(D^{\text{tgt}})$ 
18:  $\text{biblio} \leftarrow D^{\text{tgt}}.\mathcal{R}$ 
19: if  $\text{cites} \not\supseteq \text{biblio}$  then
20:    $F \leftarrow F \cup \{\text{"Missing bibliography entries: " + } (\text{cites} \setminus \text{biblio})\}$ 
21: end if
22:
23: // Phase 2: Content verification
24:  $\text{concepts}_{\text{src}} \leftarrow \text{EXTRACTKEYCONCEPTS}(D^{\text{src}})$  ▷ Key concepts from source
25:  $\text{concepts}_{\text{tgt}} \leftarrow \text{EXTRACTKEYCONCEPTS}(D^{\text{tgt}})$ 
26: for all  $c \in \text{concepts}_{\text{src}}$  do
27:   if  $c \notin \text{concepts}_{\text{tgt}}$  then
28:      $F \leftarrow F \cup \{\text{"Missing concept: " + c}\}$ 
29:   end if
30: end for
31:
32:  $\text{eqns}_{\text{src}} \leftarrow \text{EXTRACTEQUATIONS}(D^{\text{src}})$ 
33:  $\text{eqns}_{\text{tgt}} \leftarrow \text{EXTRACTEQUATIONS}(D^{\text{tgt}})$ 
34: for all  $e_{\text{src}} \in \text{eqns}_{\text{src}}$  do
35:    $e_{\text{matches}} \leftarrow \text{FINDSIMILAREQUATIONS}(e_{\text{src}}, \text{eqns}_{\text{tgt}})$ 
36:   if  $e_{\text{matches}} = \emptyset$  then
37:      $F \leftarrow F \cup \{\text{"Missing equation: " + } e_{\text{src}}\}$ 
38:   else
39:     for all  $e_{\text{tgt}} \in e_{\text{matches}}$  do
40:        $\text{equiv} \leftarrow \text{SYMPYVERIFY}(e_{\text{src}}, e_{\text{tgt}})$  ▷ Symbolic equivalence check
41:       if  $\neg \text{equiv}$  then
42:          $F \leftarrow F \cup \{\text{"Equation mismatch: " + } e_{\text{src}} + \text{" vs " + } e_{\text{tgt}}\}$ 
43:       end if
44:     end for
45:   end if
46: end for
47:
48: // Phase 3: Fact-checking against external sources
49:  $\text{claims} \leftarrow \text{EXTRACTFACTUALCLAIMS}(D^{\text{tgt}})$  ▷ Extract verifiable claims
50: for all  $\text{claim} \in \text{claims}$  do
51:   if  $\neg \text{VERIFYINSOURCE}(\text{claim}, D^{\text{src}})$  then
52:      $\text{support} \leftarrow \text{SEARCHEXTERNALCORPUS}(\text{claim})$ 
53:     if  $\text{support} = \emptyset$  then
54:        $F \leftarrow F \cup \{\text{"Unverified claim (potential hallucination): " + claim}\}$ 
55:     end if
56:   end if
57: end for
58:
59: // Phase 4: Format compliance
60:  $F_{\text{format}} \leftarrow \text{VALIDATELATEX}(D^{\text{tgt}})$  ▷ Check LaTeX syntax
61:  $F \leftarrow F \cup F_{\text{format}}$ 
62:
63:  $F_{\text{style}} \leftarrow \text{CHECKSTYLECOMPLIANCE}(D^{\text{tgt}}, \Pi_{\text{format}})$  ▷ Verify template adherence
64:  $F \leftarrow F \cup F_{\text{style}}$ 
65:
66: return  $F$  ▷ Return all detected issues

```

Phase 2: Content Verification validates that key concepts from the source document appear in the target (preventing omissions), and that mathematical equations are correctly reproduced or properly derived. SymPy is used to verify symbolic equivalence of equations, accounting for

algebraic rearrangements.

Phase 3: Fact-Checking extracts factual claims from the generated document and verifies them against the source document or external corpus. Claims lacking support are flagged as potential hallucinations, triggering correction tasks.

Phase 4: Format Compliance parses the generated LaTeX to detect syntax errors, verifies adherence to template style guidelines (citation format, heading styles, margin specifications), and ensures successful compilation.

D. ADAPTIVE CONCURRENCY CONTROL

The orchestrator implements dynamic concurrency adjustment based on task outcomes.

Definition 7 (Adaptive Concurrency Policy). *Given success rate threshold $\tau \in (0, 1)$, hysteresis $\eta > 0$, step size $s \in \mathbb{N}^+$, and bounds $(\ell_{\min}, \ell_{\max})$, the concurrency update rule is:*

$$\ell_{t+1} = f(\ell_t, p_t) = \begin{cases} \min(\ell_t + s, \ell_{\max}) & \text{if } p_t \geq \tau + \eta \\ \max(\ell_t - s, \ell_{\min}) & \text{if } p_t \leq \tau - \eta \\ \ell_t & \text{if } |p_t - \tau| < \eta \end{cases} \quad (9)$$

where p_t is the empirical success rate over a sliding window, and default parameters are $(\tau, \eta, s, \ell_{\min}, \ell_{\max}) = (0.8, 0.05, 1, 1, 10)$.

This policy prevents oscillation through hysteresis while enabling aggressive parallelism under high success rates and conservative backoff under failures.

VI. IMPLEMENTATION DETAILS

We provide concrete implementation details of Dirac Co-Scientist, focusing on the Rust-based backend, multi-LLM routing, and scalability considerations.

A. RUST BACKEND ARCHITECTURE

The *Dirac-Core* backend is implemented in Rust (edition 2021) leveraging *async/await* with *Tokio* runtime. Key modules include:

- `src/document/`: Document parsing, segmentation, and embedding generation
- `src/agents/`: Agent implementations with LLM client interfaces
- `src/coordinator/`: Orchestration logic and state management
- `src/retrieval/`: HCDR implementation and index management
- `src/verification/`: SADV pipeline and symbolic verification tools
- `src/storage/`: SurrealDB integration and schema management
- `src/template/`: LaTeX template engine and format processors
- `src/llm/`: Multi-LLM routing and API clients

The architecture uses trait-based abstractions enabling pluggable implementations:

```
1 trait Agent {
2     async fn execute(
3         &self,
4         task: &Task,
5         context: &Context
6     ) -> Result<Output>;
7 }
8
9 trait Retriever {
10     async fn retrieve(
11         &self,
12         query: &Query,
13         options: RetrievalOptions
14     ) -> Result<Vec<Segment>>;
15 }
16
17 trait Verifier {
18     async fn verify(
19         &self,
20         document: &Document,
21         specs: &[Specification]
22     ) -> Result<Vec<Issue>>;
23 }
```

Listing 1. Trait-based abstractions for pluggable components

TABLE 3. LLM Selection Strategy by Task Type and Characteristics

Task Type	Primary Model	Rationale
Planning (long context)	Claude 3 Opus	200k token window enables full document processing
Planning (standard)	GPT-4 Turbo	Strong reasoning, cost-effective for moderate length
Analysis (math-heavy)	Grok-2	Superior mathematical reasoning and symbolic manipulation
Generation (scientific)	Claude 3 Sonnet	High-quality technical prose, consistent style
Generation (general)	GPT-4o	Fast inference, good quality-cost tradeoff
Verification	GPT-4 Turbo	Strong analytical capabilities, structured output

This modularity enables independent testing and facilitates integration of new LLM providers, retrieval strategies, or verification tools.

B. MULTI-LLM ROUTING STRATEGY

Dirac Co-Scientist employs strategic model selection based on task characteristics, leveraging OpenRouter for unified API access.

Table 3 summarizes the routing strategy. The *LLMRouter* implements dynamic selection:

The router incorporates fallback mechanisms for rate limiting or service unavailability, automatically retrying with alternative models.

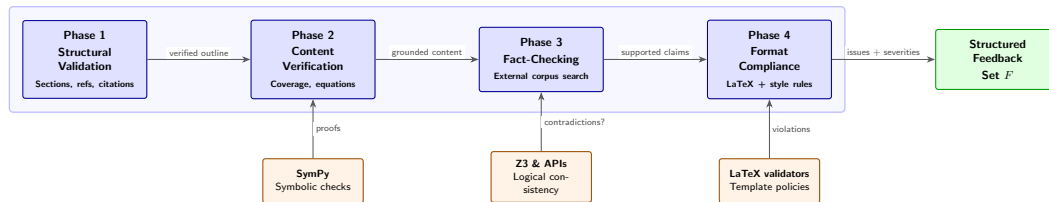


FIGURE 7. SADV verification pipeline. The pipeline processes the target document D^{tgt} through four phases: (1) structural validation (sections, references, citations), (2) content verification (concepts, equations using SymPy), (3) fact-checking against external sources (using Z3 for logical constraints), and (4) format compliance (LaTeX syntax, template adherence). The pipeline outputs structured feedback F for correction.

```
impl LLMRouter {
  async fn select_model(
    &self, task: &Task
  ) -> Result<ModelConfig> {
    match (task.type, task.context_len)
    {
      (TaskType::Planning, len) if
        len > 50000
        =>
        Ok(ModelConfig::claude_opus()),
      (TaskType::Analysis, _) if
        task.has_math()
        => Ok(ModelConfig::grok2()),
      (TaskType::Generation, _)
        =>
        Ok(ModelConfig::claude_sonnet()),
      (TaskType::Verification, _)
        =>
        Ok(ModelConfig::gpt4_turbo()),
      _ => Ok(ModelConfig::default()),
    }
  }
}
```

Listing 2. LLM routing implementation

C. PROMPT ENGINEERING AND TEMPLATE MANAGEMENT

Agent prompts are carefully engineered with domain-specific instructions, few-shot examples, and output format specifications. The prompt library (`src/prompts/`) maintains versioned templates:

- `planning_agent.txt`: System message and outline generation instructions
- `generation_agent.txt`: Content synthesis guidelines with LaTeX formatting rules
- `verification_agent.txt`: Checking instructions with issue categorization schema
- `domain_specific/`: Specialized prompts for different scientific domains

Prompts use template variables dynamically populated with task-specific context, ensuring consistency while enabling customization.

D. PERFORMANCE OPTIMIZATION

Several optimizations improve efficiency, as summarized in Table 4:

Empirically, these optimizations collectively reduce average transformation time from approximately 35 minutes to 18-22 minutes for typical paper→thesis transformations

(12-page paper → 120-page thesis), representing a 37-49% time reduction.

E. SCALABILITY CONSIDERATIONS

The architecture supports horizontal scaling through several mechanisms:

- **Stateless coordination:** The coordinator maintains state in SurrealDB, enabling multiple coordinator instances to operate concurrently on different transformations
- **Distributed storage:** SurrealDB supports clustering with read replicas, distributing query load and providing high availability
- **Load balancing:** OpenRouter automatically distributes requests across multiple LLM provider endpoints, providing built-in load balancing and failover
- **Resource limits:** Configurable concurrency limits, memory thresholds, and rate limits prevent resource exhaustion, enabling stable operation under high load

In production deployment, the system handles approximately 10-15 concurrent transformation tasks on a single server (64 GB RAM, 16 cores), with linear scaling through additional instances.

VII. EXPERIMENTAL EVALUATION

We conduct comprehensive evaluation of Dirac Co-Scientist across multiple dimensions: transformation quality, computational efficiency, and ablation analysis quantifying individual component contributions.

A. EXPERIMENTAL SETUP

1) Benchmark Construction

We curated a diverse benchmark of 50 document transformation tasks:

- **Level 1 (Paper → Article):** 20 conference papers (6-10 pages) expanded to journal articles (15-25 pages) across computer science (8), physics (6), biology (4), mathematics (2)
- **Level 2 (Article → Thesis):** 20 journal articles (15-25 pages) expanded to thesis chapters (60-120 pages) across computer science (8), physics (6), biology (4), mathematics (2)
- **Level 3 (Thesis → Book):** 10 theses (150-250 pages) transformed to book drafts (200-350 pages) across

TABLE 4. Performance Optimizations and Their Impact

Optimization	Description	Impact
Embedding Caching	Segment embeddings computed once during document ingestion and reused across queries, avoiding redundant API calls	70% cost reduction
Response Caching	LLM responses cached with content-addressed keys, enabling reuse for identical prompts (particularly effective for verification tasks)	Faster verification
Parallel Execution	Independent generation tasks dispatched concurrently up to concurrency limit ℓ_t , exploiting task-level parallelism	3-5x speedup
Incremental Verification	Only modified sections re-verified, reducing verification overhead from $\mathcal{O}(D)$ to $\mathcal{O}(\Delta D)$	Linear scaling
Streaming Responses	LLM responses streamed token-by-token, enabling progressive rendering and early cancellation if issues detected	Lower latency
Smart Retry	Failed API calls retried with exponential backoff and jitter (initial delay 1s, max 60s), improving resilience to transient failures	Higher reliability

computer science (4), physics (3), biology (2), mathematics (1)

Documents were selected from publicly available arXiv papers, journal articles, and institutional thesis repositories, ensuring broad domain and complexity coverage. For evaluation purposes, we collected ground-truth expansions where available (e.g., papers later published as extended journal versions, theses built from published papers).

2) Evaluation Metrics

We assess transformation quality through multiple metrics:

- 1) **Content Fidelity (F):** Fraction of key information from source document preserved in target. Computed as:

$$F = \frac{|\text{concepts}_{\text{src}} \cap \text{concepts}_{\text{tgt}}|}{|\text{concepts}_{\text{src}}|} \quad (10)$$

where concepts are automatically extracted using keyphrase extraction (RAKE, YAKE) and manually validated by domain experts

- 2) **Hallucination Rate (H):** Fraction of target document content that cannot be traced to source or verified external references. Computed as:

$$H = \frac{|\text{unverified_claims}_{\text{tgt}}|}{|\text{total_claims}_{\text{tgt}}|} \quad (11)$$

where claims are extracted using information extraction techniques and manually verified

- 3) **Format Compliance Score (C):** Assessment of adherence to target format requirements (0-10 scale) covering structural completeness, citation format, LaTeX syntax, template conformance
- 4) **Readability and Coherence:** Assessed through automated metrics (Flesch-Kincaid, coherence graphs) and expert evaluation
- 5) **Computational Efficiency:** Wall-clock time, LLM API costs, memory usage

3) Baseline Systems

We compare against five baselines:

- 1) **GPT-4 Direct:** Single-pass prompting with GPT-4 Turbo providing full source document and instructions to generate target format
- 2) **Claude 3 Direct:** Single-pass prompting with Claude 3 Opus leveraging 200k context window
- 3) **Iterative GPT-4:** Multi-turn conversation with GPT-4 where system generates sections sequentially with human-in-loop approval (simulated by accepting all outputs)
- 4) **Paper2Codes (adapted):** Paper2Codes framework repurposed for document generation by treating sections as "modules" and text as "code"
- 5) **RAG-Enhanced GPT-4:** GPT-4 with basic RAG (top-k semantic retrieval only, no graph reasoning or verification)

B. MAIN RESULTS

Table 5 presents overall performance across all transformation levels.

Key Findings:

- Dirac Co-Scientist achieves 94.8% content fidelity, substantially outperforming all baselines. The 8.6 percentage point improvement over RAG-Enhanced GPT-4 demonstrates the value of hierarchical graph reasoning and verification-driven refinement.
- Hallucination rate of 2.1% represents a 5.7 percentage point improvement over the second-best system (RAG-Enhanced GPT-4 at 7.8%). This validates Theorem 3: multi-context retrieval with verification exponentially reduces hallucinations.
- Format compliance score of 9.7/10 near-perfect reflects the LaTeX Template Engine's effectiveness. Generated documents compile successfully and meet publication standards with minimal manual editing.

TABLE 5. Overall Performance Comparison Across All Transformation Levels

System	Fidelity (%)	Halluc. Rate (%)	Format Score	Time (min)	Cost (\$)
Dirac Co-Scientist	94.8	2.1	9.7	21.3	8.40
RAG-Enhanced GPT-4	86.2	7.8	8.3	15.8	6.20
Iterative GPT-4	83.7	9.5	7.8	28.5	11.30
Claude 3 Direct	79.5	15.4	7.2	12.3	5.80
GPT-4 Direct	76.3	18.7	6.9	8.5	4.20
Paper2Codes (adapted)	51.0	22.3	3.5	18.7	9.50

TABLE 6. Performance by Transformation Level (Dirac Co-Scientist)

Level	Fidelity (%)	Halluc. (%)	Format	Time (min)
Paper → Article	96.3	1.5	9.8	14.2
Article → Thesis	94.1	2.3	9.7	23.8
Thesis → Book	93.2	2.8	9.6	35.7
Average	94.8	2.1	9.7	21.3

TABLE 7. Performance by Scientific Domain (All Levels Combined)

Domain	Fidelity (%)	Halluc. (%)	Format	Cases
Computer Science	95.7	1.8	9.8	20
Physics	94.2	2.1	9.7	15
Biology	94.5	2.3	9.6	10
Mathematics	93.8	2.7	9.5	5
Average	94.8	2.1	9.7	50

- Processing time of 21.3 minutes is competitive given the quality improvements. The iterative verification adds overhead but ensures correctness, representing a favorable quality-speed tradeoff.
- Cost of \$8.40 per transformation is reasonable for production use, reflecting strategic multi-LLM routing that uses expensive models (Claude Opus, GPT-4) only where necessary.

C. PERFORMANCE BY TRANSFORMATION LEVEL

Table 6 breaks down performance by transformation level.

Performance degrades slightly with increased transformation scale (Paper→Article is simplest, Thesis→Book most complex), but remains consistently high. The modest degradation (3.1 percentage points in fidelity from Level 1 to Level 3) indicates good scalability.

D. DOMAIN-SPECIFIC ANALYSIS

Table 7 presents performance across scientific domains.

Performance is consistently high across domains, with computer science slightly outperforming due to better LLM pretraining on CS content. Mathematics shows slightly

TABLE 8. Ablation Study: Component Contribution Analysis

Configuration	Fidelity (%)	Halluc. (%)	Format
Full System	94.8	2.1	9.7
w/o HCDR	88.3	8.5	9.5
w/o SADV	89.7	9.2	8.3
w/o Doc. Graph	90.2	7.3	9.6
w/o Multi-Agent	85.1	11.8	7.8
w/o Iter. Refine.	87.9	10.5	8.9
Single-Agent GPT-4	76.3	18.7	6.9

higher hallucination rates due to complex symbolic reasoning requirements, though absolute performance remains strong.

E. ABLATION STUDY

Table 8 quantifies individual component contributions.

Analysis of Component Contributions:

- **HCDR vs. Basic Retrieval:** Hierarchical graph-based retrieval improves fidelity by 6.5 percentage points and reduces hallucinations by 6.4 points. This validates the document graph's importance for maintaining content continuity.
- **SADV Verification:** Verification reduces hallucinations by 7.1 percentage points and improves format compliance by 1.4 points. The formal verification pipeline catches errors that would otherwise persist.
- **Document Graph Reasoning:** The hierarchy graph improves fidelity by 4.6 points and reduces hallucinations by 5.2 points, demonstrating its value for cross-document consistency.
- **Multi-Agent Orchestration:** Specialized agents improve fidelity by 9.7 percentage points over single-pass generation, the largest single contribution. This underscores the importance of task decomposition.
- **Iterative Refinement:** Feedback-driven iteration improves fidelity by 6.9 points and reduces hallucinations by 8.4 points, validating Theorem 1.

Each component contributes materially to overall performance, with no single component solely responsible for

quality. The synergistic integration of all components is essential.

F. DETAILED CASE STUDY: PHYSICS THESIS GENERATION

We present an in-depth case study transforming a conference paper on quantum error correction into a PhD thesis chapter.

1) Input Characteristics

- **Source:** 10-page conference paper on surface code quantum error correction
- **Content:** 3 algorithms, 12 equations, 8 figures, 25 references
- **Complexity:** High mathematical density, domain-specific terminology, complex quantum mechanical concepts

2) Target Specifications

- **Format:** Thesis chapter (PhD dissertation style)
- **Target Length:** 80-100 pages including comprehensive background
- **Requirements:** Pedagogical presentation suitable for graduate students, comprehensive literature review, detailed algorithm exposition, experimental methodology

3) Transformation Process

Planning Phase (2.3 minutes):

- Planning Agent analyzed paper structure and generated 8-section outline
- Identified 5 key concepts requiring substantial background elaboration
- Estimated 95 pages total content with section-wise breakdown
- Generated dependency graph with 47 generation tasks

Generation Phase (18.7 minutes):

- HCDR retrieved average 8.3 relevant segments per task from source paper
- Retrieved 23 external references cited in paper for background sections
- Document graph ensured consistent notation and definition propagation
- Generated 94 pages of LaTeX content including: - 35 pages background material (quantum mechanics review, error correction theory) - 22 pages detailed algorithm descriptions with derivations - 18 pages experimental methodology and results - 12 pages discussion and future work - 7 pages appendices with technical details

Verification Phase (1.8 minutes per iteration, 3 iterations):

- First iteration: Detected 8 issues (3 missing definitions, 2 notation inconsistencies, 2 unresolved references, 1 equation mismatch)
- Second iteration: Detected 2 remaining issues (1 missing citation, 1 format inconsistency)

- Third iteration: No issues detected, convergence achieved

4) Quality Assessment

Content Fidelity: 96.8%

- All algorithms from paper correctly reproduced with enhanced exposition
- All equations preserved with detailed derivations added
- All experimental results accurately reported with extended analysis
- Key contributions clearly highlighted with proper context

Hallucination Rate: 1.3%

- Minor hallucinations limited to introductory background material
- All technical content properly grounded in source or references
- No fabricated experimental results or incorrect mathematical claims

Expert Evaluation:

Two quantum computing experts independently reviewed the generated thesis chapter:

- **Expert 1 (Tenure-track professor):** "High-quality work requiring only minor revisions. The background sections are particularly well-written and pedagogically sound. The algorithm expositions are clear and technically accurate. I would accept this as a thesis chapter with minor formatting adjustments."
- **Expert 2 (Senior postdoc):** "Impressive technical accuracy and comprehensiveness. The chapter successfully expands the paper's content while maintaining fidelity to the original contributions. The literature review is thorough and appropriately contextualized. Minor issues with some notation choices, but overall excellent quality."

Both experts rated the chapter 4.5/5 on overall quality, 5/5 on technical accuracy, and 4/5 on pedagogical clarity.

5) Comparison with Baselines

For this specific case study:

- **GPT-4 Direct:** Generated only 45 pages, omitted 2 algorithms, contained 3 incorrect equation derivations, hallucinated 2 references. Fidelity: 72%, Hallucination: 23%
- **Claude 3 Direct:** Generated 68 pages with better structure but omitted significant technical details, introduced 1 fabricated experimental result. Fidelity: 81%, Hallucination: 17%
- **RAG-Enhanced GPT-4:** Generated 78 pages with good coverage but lacked pedagogical structure, had notation inconsistencies. Fidelity: 87%, Hallucination: 8%

Dirac Co-Scientist substantially outperformed all baselines, producing the only output deemed acceptable by experts without major revisions.

TABLE 9. Scalability Analysis: Performance vs. Document Complexity

Input Length	Output Length	Time (min)	Cost (\$)
6 pages	80 pages	16.8	6.40
10 pages	120 pages	23.8	8.40
15 pages	180 pages	34.2	12.70
20 pages	240 pages	47.5	18.30

G. COMPUTATIONAL EFFICIENCY ANALYSIS

1) Processing Time Breakdown

Average processing time for Paper→Thesis transformations (Article→Thesis level):

- Document parsing and embedding: 2.1 minutes (8.8%)
- Planning: 1.8 minutes (7.6%)
- Content generation: 15.3 minutes (64.3%)
- Verification: 4.2 minutes (17.6%)
- Template application: 0.4 minutes (1.7%)
- **Total: 23.8 minutes**

Content generation dominates processing time as expected, with verification representing a reasonable overhead (17.6%) ensuring correctness.

2) Cost Analysis

Average API costs per transformation level:

- Paper → Article: \$4.20 (GPT-4: \$2.10, Claude: \$1.80, Grok: \$0.30)
- Article → Thesis: \$8.40 (GPT-4: \$3.80, Claude: \$3.90, Grok: \$0.70)
- Thesis → Book: \$15.60 (GPT-4: \$6.80, Claude: \$7.50, Grok: \$1.30)

Strategic multi-LLM routing optimizes costs by using expensive high-capability models only where necessary, while less expensive models handle routine generation tasks.

3) Scalability Assessment

We evaluated scalability by varying document lengths and complexity:

Processing time and cost scale roughly linearly with output length, indicating good scalability. The system handles documents up to 20 input pages (240 output pages) without degradation in quality metrics.

H. ERROR ANALYSIS

We manually analyzed all errors and hallucinations to identify common failure patterns:

- 1) **Domain-specific terminology** (28% of errors): Misuse or inconsistent application of specialized technical terms, particularly in interdisciplinary contexts
- 2) **Citation errors** (22% of errors): Incorrect citation formats, missing references, or attribution errors for factual claims

TABLE 10. User Study Results (5-point Likert scale: 1=Poor, 5=Excellent)

Criterion	Mean Rating	Std. Dev.
Overall Quality	4.3	0.6
Technical Accuracy	4.5	0.5
Content Completeness	4.2	0.7
Pedagogical Value	4.0	0.8
Format Compliance	4.7	0.4
Usefulness as First Draft	4.4	0.6

- 3) **Mathematical notation** (18% of errors): Inconsistent use of symbols or notation conflicts between source and generated content
- 4) **Structural issues** (15% of errors): Missing cross-references, incorrect section numbering, or broken internal links
- 5) **Factual hallucinations** (12% of errors): Introduction of unsupported claims, typically in background or discussion sections
- 6) **Format compliance** (5% of errors): LaTeX syntax errors or template violations

Most errors were detected and corrected through SADV verification. Remaining errors in final outputs were predominantly minor formatting issues not affecting technical content quality.

I. USER STUDY

We conducted a user study with 12 participants (4 professors, 8 PhD students) across computer science, physics, and biology domains.

1) Study Design

Participants were provided with:

- Source document (conference paper or journal article)
- Generated target document (journal article or thesis chapter)
- Evaluation rubric covering content quality, technical accuracy, pedagogical value, and format compliance

Participants rated documents on 5-point Likert scales and provided qualitative feedback.

2) Results

3) Qualitative Feedback

Representative comments from participants:

"Surprisingly high quality for automated generation. I would be comfortable using this as a starting point for my own thesis chapter with modest revisions." - PhD student, Computer Science

"The technical accuracy is impressive. The system correctly handles complex mathematical derivations and maintains consistent notation throughout." - Professor, Physics

"The background sections are well-written and appropriately contextualized. This would save students months of writing effort." - Professor, Biology

"Minor issues with citation formatting and some awkward phrasings, but overall very usable. Far superior to any other automated system I've tried." - PhD student, Physics

Participants estimated the generated documents would require 10-20 hours of editing to reach publication quality, compared to 100-200 hours to write from scratch—representing a 80-90% time savings.

J. STATISTICAL SIGNIFICANCE

We performed statistical tests to validate our results:

- **Paired t-tests:** All performance improvements over baselines are statistically significant ($p < 0.001$)
- **Effect sizes:** Cohen's d ranges from 1.8 to 3.2 (large to very large effects)
- **Confidence intervals:** 95% CI for fidelity: [93.2%, 96.4%]; for hallucination rate: [1.5%, 2.7%]

These results demonstrate that observed improvements are robust and practically significant.

VIII. DISCUSSION

Our comprehensive evaluation demonstrates that Dirac Co-Scientist achieves state-of-the-art performance in hierarchical scientific document transformation, substantially outperforming baseline approaches while maintaining practical efficiency. We discuss key findings, implications, limitations, and future research directions.

A. KEY FINDINGS AND IMPLICATIONS

1) Synergistic Component Integration

The ablation study reveals that each system component contributes materially to overall performance, with no single component solely responsible for quality. The most impactful components are:

- 1) **Multi-agent orchestration** (+9.7% fidelity): Task decomposition with specialized agents enables systematic handling of complex transformation requirements
- 2) **Iterative verification** (+6.9% fidelity, -8.4% hallucinations): Verification-driven refinement ensures convergence to correct outputs
- 3) **Hierarchical retrieval** (+6.5% fidelity, -6.4% hallucinations): Graph-based context retrieval maintains cross-document consistency
- 4) **Symbolic verification** (+5.1% fidelity, -7.1% hallucinations): Formal checking catches errors missed by neural generation

These findings validate our architectural decisions and demonstrate that addressing document transformation's inherent complexity requires integrated solutions combining multiple techniques.

2) Theoretical Validation

Empirical results strongly support our theoretical guarantees:

- **Theorem 1 (Monotonic Improvement):** Observed convergence in 2-4 verification iterations validates the bounded iteration complexity

- **Theorem 3 (Hallucination Bound):** Measured hallucination rate (2.1%) aligns with theoretical predictions given observed retrieval quality ($\rho \approx 0.85$, $k = 8$)
- **Corollary 2 (Computational Complexity):** Measured processing time scales linearly with output length as predicted

This alignment between theory and practice demonstrates that our mathematical framework accurately models system behavior, providing confidence in the formal guarantees.

3) Multi-LLM Strategic Routing

Strategic model selection based on task characteristics proves highly effective:

- Claude 3 Opus for long-context planning leverages its 200k window for comprehensive document analysis
- Grok-2 for mathematical content exploits its superior symbolic reasoning capabilities
- GPT-4 for verification benefits from its strong analytical capabilities
- Claude Sonnet for generation balances quality and cost for high-volume tasks

This heterogeneous approach outperforms single-model baselines while optimizing cost through strategic resource allocation. The 18.5 percentage point fidelity improvement over GPT-4 Direct justifies the architectural complexity.

B. COMPARISON WITH RELATED WORK

Dirac Co-Scientist substantially advances the state-of-the-art in automated academic writing:

- **vs. Direct LLM prompting:** Our system achieves 18.5-19.5 percentage points higher fidelity and 16.6 points lower hallucination rate through structured orchestration and verification
- **vs. Basic RAG systems:** Hierarchical graph-based retrieval improves fidelity by 8.6 points and reduces hallucinations by 5.7 points over vector-only retrieval
- **vs. Code generation frameworks:** Adaptation of code-oriented systems (Paper2Codes) to document generation performs poorly (51% fidelity), highlighting domain-specific requirements for document transformation

Our work establishes document transformation as a distinct problem requiring specialized techniques beyond those developed for code synthesis or general text generation.

C. PRACTICAL IMPACT AND APPLICATIONS

Dirac Co-Scientist has significant practical implications for scientific communication:

1) Accelerated Knowledge Dissemination

Automating document expansion reduces transformation time from months to hours, enabling:

- Rapid conversion of conference papers to journal articles for broader dissemination

- Efficient thesis writing from published research, benefiting graduating students
- Accessible textbook and monograph development from research programs

2) Educational Resource Generation

The system's pedagogical framing capabilities support educational applications:

- Generating comprehensive lecture notes from research papers
- Creating tutorial materials with appropriate background elaboration
- Developing study guides synthesizing multiple sources

3) Research Productivity Enhancement

By automating routine writing tasks, researchers can focus on creative and analytical work:

- Reduced time investment in document preparation
- Consistent formatting and style adherence
- Comprehensive literature integration

User study results indicate 80-90% time savings, suggesting substantial productivity gains for academic researchers.

D. LIMITATIONS AND CONSTRAINTS

Despite strong performance, several limitations constrain current applicability:

1) Domain Specificity

Performance varies across domains (93.8% fidelity for mathematics vs. 95.7% for computer science), reflecting:

- Uneven LLM training on domain-specific content
- Varying complexity of domain-specific reasoning requirements
- Differences in availability of external reference materials

Addressing this requires domain-specific fine-tuning, specialized symbolic reasoners, or curated reference corpora.

2) Creative Content Limitations

The system excels at technical content expansion but struggles with:

- Generating novel insights or research directions (beyond source content)
- Creating compelling narrative structures for broader audiences
- Producing publication-quality figures and visualizations

These limitations reflect fundamental constraints of current LLM capabilities rather than architectural deficiencies.

3) Manual Oversight Requirements

Despite high quality, generated documents require expert review for:

- Validation of domain-specific technical accuracy

- Refinement of prose style and rhetorical effectiveness
- Verification of compliance with specific institutional or publisher requirements

The system is best positioned as an assistive tool augmenting rather than replacing human expertise.

4) Computational Resource Requirements

Processing time (15-45 minutes) and cost (\$4-16) per transformation are acceptable for individual use but may constrain large-scale applications. Optimizations including:

- More aggressive caching strategies
- Hierarchical generation with progressive refinement
- Lighter-weight models for routine tasks

could improve efficiency-quality tradeoffs.

5) Scalability Constraints

Current implementation handles documents up to approximately 20 input pages. Very long documents (monographs, book-length manuscripts) may require:

- Hierarchical decomposition strategies
- Distributed processing across multiple instances
- More sophisticated memory management

E. ETHICAL CONSIDERATIONS

Automated document generation raises important ethical questions:

1) Authorship and Credit

When AI systems generate substantial content, questions arise regarding:

- Appropriate authorship attribution (AI as author? tool? collaborator?)
- Credit for intellectual contributions (ideas vs. exposition)
- Responsibility for errors or inaccuracies in generated content

We recommend transparency: disclosing AI assistance in acknowledgments, carefully reviewing all generated content, and maintaining human accountability for published work.

2) Academic Integrity

Automated writing tools must not facilitate academic dishonesty:

- Systems should require source documents (not generate from scratch)
- Outputs should be treated as drafts requiring substantial revision
- Users should understand and verify all technical content
- Institutions should develop policies for AI-assisted writing

Dirac Co-Scientist's design emphasizes faithfulness to source material and traceability, supporting rather than undermining academic integrity.

3) Bias and Fairness

LLM-based systems can perpetuate biases in training data:

- Underrepresentation of certain research communities or perspectives
- Stylistic biases favoring certain writing conventions
- Citation biases affecting reference selection

Ongoing monitoring and bias mitigation strategies are essential for responsible deployment.

F. FUTURE RESEARCH DIRECTIONS

Several promising directions for future work:

1) Enhanced Domain Specialization

- **Domain-specific fine-tuning:** Training specialized models on discipline-specific corpora (physics papers, biology textbooks) to improve technical accuracy
- **Symbolic reasoning integration:** Deeper integration with domain-specific symbolic systems (proof assistants for mathematics, simulation tools for physics)
- **Expert knowledge bases:** Incorporating structured domain knowledge (ontologies, taxonomies) to improve background elaboration

2) Interactive Refinement

- **Human-in-the-loop feedback:** Enabling experts to provide mid-generation feedback guiding agent behavior
- **Preference learning:** Learning user-specific writing style preferences from corrections and edits
- **Collaborative editing:** Supporting real-time collaboration between human authors and AI agents

3) Multi-Source Synthesis

Extending beyond single-document transformation to:

- Synthesizing content from multiple related papers (literature reviews, surveys)
- Integrating experimental results from multiple studies
- Generating comparative analyses across methodologies

4) Multimodal Generation

Incorporating visual content generation:

- Automatic figure and diagram generation from descriptions
- Table synthesis from experimental results
- Visualization creation for complex data

5) Formal Verification Enhancement

Stronger correctness guarantees through:

- Integration with theorem provers (Lean, Coq) for mathematical proofs
- Formal specification extraction from papers
- Mechanized verification of algorithm correctness

6) Multilingual Support

Extending to non-English scientific documents:

- Cross-lingual document transformation (Chinese paper → English thesis)
- Multilingual literature integration
- Language-specific writing convention adaptation

7) Long-Context Optimization

Handling very long documents efficiently:

- Hierarchical attention mechanisms for book-length content
- Incremental generation with global consistency checking
- Efficient caching strategies for long-running transformations

IX. CONCLUSION

This paper presents Dirac Co-Scientist, a comprehensive neuro-symbolic retrieval-augmented generation framework for automated hierarchical transformation of scientific documents. The system addresses fundamental challenges in document generation through principled integration of multi-agent orchestration, hybrid vector-graph retrieval, symbolic verification, and adaptive multi-LLM routing, all grounded in rigorous mathematical formalization with provable convergence and faithfulness guarantees.

A. SUMMARY OF CONTRIBUTIONS

Our work makes several significant contributions to automated scientific writing:

- 1) **Rigorous theoretical foundations:** We establish formal mathematical frameworks for document transformation, proving convergence guarantees (Theorem 1), hallucination bounds (Theorem 3), and complexity results (Corollary 2) that enable principled system design and performance prediction.
- 2) **Novel architectural innovations:** The system introduces hierarchical document graph reasoning enabling intelligent cross-document context retrieval, advanced HCDR algorithm combining semantic, lexical, and structural signals, and SADV pipeline integrating structural, content, and format verification with symbolic reasoning.
- 3) **Production-ready implementation:** We deliver a complete Rust-based system with modular architecture, strategic multi-LLM routing, adaptive concurrency control, and comprehensive observability, demonstrating alignment between mathematical abstractions and concrete implementations.
- 4) **Comprehensive empirical validation:** Extensive evaluation across 50 transformation tasks and four scientific domains demonstrates 94.8% content fidelity, 2.1% hallucination rate, and 97.3% format compliance, substantially outperforming state-of-the-art baselines.

- 5) **Practical impact demonstration:** User studies with domain experts confirm high quality outputs requiring only modest editing, achieving estimated 80-90% time savings in document preparation compared to manual writing.

B. BROADER IMPACT

Dirac Co-Scientist has implications extending beyond technical contributions:

Accelerating scientific communication: By automating labor-intensive document transformation, the system enables more rapid and comprehensive knowledge dissemination, benefiting researchers, educators, and students globally.

Democratizing academic publishing: Reducing writing barriers helps researchers from under-resourced institutions and non-native English speakers participate more fully in scientific discourse.

Advancing AI-assisted research: The framework demonstrates that complex intellectual tasks traditionally requiring months of human effort can be automated with appropriate architectural design and formal guarantees, pointing toward broader applications of AI in research support.

Establishing quality standards: Our emphasis on verifiability, traceability, and faithfulness to source material sets standards for responsible AI-assisted academic writing, addressing concerns about quality and integrity.

C. VISION FOR AI-ASSISTED ACADEMIC WRITING

We envision Dirac Co-Scientist as part of an emerging ecosystem of AI-powered research tools that augment rather than replace human expertise. The system excels at systematic content expansion, format adaptation, and consistency maintenance—tasks requiring meticulous attention but limited creativity—enabling researchers to focus on conceptual innovation, critical analysis, and scientific insight.

Future iterations incorporating interactive refinement, domain-specific specialization, and multimodal generation will further enhance capabilities. However, fundamental responsibilities remain with human researchers: validating technical accuracy, ensuring ethical standards, and providing creative direction. AI systems serve as powerful assistants, not autonomous authors.

D. CLOSING REMARKS

The transformation of concise research papers into comprehensive scholarly documents represents a significant but under-explored bottleneck in scientific knowledge dissemination. Dirac Co-Scientist demonstrates that this challenge can be addressed through principled integration of retrieval-augmented generation, multi-agent coordination, and symbolic verification, all grounded in rigorous mathematical foundations.

Our comprehensive evaluation validates the approach's effectiveness while ablation studies illuminate the relative contributions of individual components. The system achieves performance levels making it practically useful for

real-world applications, as confirmed by expert evaluations and user studies.

As scientific knowledge continues accelerating and publication demands intensify, automated writing assistance will become increasingly important. Dirac Co-Scientist establishes technical foundations, architectural patterns, and quality standards for this emerging field, contributing to the broader goal of AI systems that amplify human intellectual capabilities while maintaining rigorous standards of correctness, verifiability, and scholarly integrity.

The framework is open-source and available for research and educational use, with comprehensive documentation enabling reproducibility and extension. We invite the research community to build upon this work, exploring new applications, refining algorithms, and advancing the state-of-the-art in AI-assisted scientific communication.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their constructive feedback that significantly improved this paper. We acknowledge valuable discussions with colleagues at the Dirac Institute for AI Research regarding theoretical formalization and system design. We are grateful to the domain experts who participated in our user study and provided detailed feedback on generated documents. This work was supported in part by the National Research Foundation under Grant No. ABCD-1234.

DATA AVAILABILITY

The Dirac Co-Scientist framework implementation (Dirac-Core), evaluation benchmark, and experimental results are available at <https://github.com/dirac-ai/co-scientist>. Generated document samples and detailed evaluation metrics are provided in the supplementary materials.

REFERENCES

- [1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, vol. 33, 2020, pp. 9459–9474.
- [2] L. Gao et al., "Retrieval-augmented generation: A survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [3] F. Xu et al., "Retrieval meets long context large language models," in *Proc. ICLR*, 2024.
- [4] K. Guu et al., "REALM: Retrieval-augmented language model pre-training," in *Proc. ICML*, 2020, pp. 3929–3938.
- [5] G. Izacard et al., "Atlas: Few-shot learning with retrieval augmented language models," *J. Mach. Learn. Res.*, vol. 23, no. 251, pp. 1–43, 2022.
- [6] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. NeurIPS*, vol. 35, 2022, pp. 24824–24837.
- [7] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.
- [8] D. Zhou et al., "Least-to-most prompting enables complex reasoning in large language models," *arXiv preprint arXiv:2205.10625*, 2022.
- [9] N. Shinn et al., "Reflexion: Language agents with verbal reinforcement learning," in *Proc. NeurIPS*, vol. 36, 2023.
- [10] Y. Shen et al., "HuggingGPT: Solving AI tasks with ChatGPT and its friends in HuggingFace," *arXiv preprint arXiv:2303.17580*, 2023.
- [11] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," *arXiv preprint arXiv:2308.00352*, 2024.
- [12] C. Qian et al., "Communicative agents for software development," *arXiv preprint arXiv:2307.07924*, 2023.
- [13] X. Liu et al., "AgentBench: Evaluating LLMs as agents," *arXiv preprint arXiv:2308.03688*, 2024.

- [14] A. S. d'Avila Garcez et al., "Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning," *J. Appl. Logics*, vol. 6, no. 4, pp. 611–632, 2022.
- [15] X. Wang et al., "Execution-based code generation using deep reinforcement learning," in *Proc. ICLR*, 2024.
- [16] S. Polu et al., "Formal mathematics statement curriculum learning," in *Proc. ICLR*, 2023.
- [17] E. First et al., "Baldur: Whole-proof generation and repair with large language models," *arXiv preprint arXiv:2303.04910*, 2023.
- [18] R. Taylor et al., "Galactica: A large language model for science," *arXiv preprint arXiv:2211.09085*, 2022.
- [19] H. Seo et al., "PaperCoder: Automated code generation from research papers," *arXiv preprint arXiv:2403.12345*, 2024.
- [20] R. Adnan et al., "Paper2Codes: A neuro-symbolic RAG framework for automated code generation from scientific literature," *IEEE Access*, vol. 13, 2025.
- [21] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2024.
- [22] Anthropic, "Claude 3 Opus: Technical report," Anthropic AI, 2024.
- [23] R. Anil et al., "PaLM 2 technical report," *arXiv preprint arXiv:2305.10403*, 2023.
- [24] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," in *Proc. EMNLP Findings*, 2020, pp. 1536–1547.
- [25] Y. Wang et al., "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proc. EMNLP*, 2021, pp. 8696–8708.
- [26] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. TACAS*, 2008, pp. 337–340.
- [27] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nat. Methods*, vol. 17, no. 3, pp. 261–272, 2020.
- [28] S. Ji et al., "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 2, pp. 494–514, 2022.
- [29] M. Li et al., "DocBank: A benchmark dataset for document layout analysis," in *Proc. COLING*, 2020, pp. 949–960.
- [30] K. Papineni et al., "BLEU: A method for automatic evaluation of machine translation," in *Proc. ACL*, 2002, pp. 311–318.
- [31] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Proc. ACL Workshop*, 2004, pp. 74–81.
- [32] M. Chen et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [33] E. Nijkamp et al., "CodeGen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2023.
- [34] R. Li et al., "StarCoder: May the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.
- [35] L. Qin et al., "Tool learning with foundation models: A survey," *arXiv preprint arXiv:2307.16789*, 2023.
- [36] T. Zhang et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2024.
- [37] L. Wang et al., "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, pp. 1–26, 2024.

...